

Large Language Models (LLMs) have reshaped the mobile application ecosystem in barely two years. Developers now routinely embed conversational agents, generative writing tools, and intelligent assistants into consumer apps [7], and the pace shows no sign of slowing. For the platforms and regulators that host these apps, the practical question is no longer *whether* apps integrate LLMs but *which ones do* — a question that surfaces directly in data-privacy auditing, content-safety review, and emerging AI-regulation compliance [1, 8, 35]. The bottleneck is throughput: at Google Play scale, any review pipeline has to triage millions of candidates before a human or an APK-level tool ever looks closely.

The dominant detection approach today is to peer inside the binary. The AI Discriminator tool decompiles Android Packages (APKs) and matches keywords against a curated dictionary of ML and DL library identifiers [14]; the more recent LLMAID framework replaces those keyword heuristics with an LLM that semantically inspects decompiled code [15]. Both achieve high precision on apps for which an APK is available. Neither, however, makes a comfortable first-pass filter: each requires APK access, scales linearly in expensive analysis steps, and is brittle to code obfuscation, dynamic API loading, and the increasingly common pattern of off-device LLM calls that leave little to no static footprint [5, 21, 29]. The result is a gap in the pipeline: code-level tools want to be *deep* inspectors, but the ecosystem also needs a *wide* screener that runs upstream of them.

Public app store listings — titles, full descriptions, promotional screenshots — are the obvious place to look. They are user-facing by design, freely available without APK access, and prior work shows that combining their textual and visual channels improves general app-classification tasks [4, 26, 27]. Yet listing-only LLM-integration detection remains underexplored: app-store studies have mostly emphasized text or user reviews [6], while multimodal LLM (MLLM) research on mobile has gone in adjacent directions — UI understanding, bug detection, deceptive-pattern auditing [9, 13, 25]. We aim to close this gap.

This paper proposes **LLMDroid**, a multimodal screening framework that flags likely LLM-integrated mobile apps from publicly available metadata alone. We frame the problem as *proxy-evidence screening*: the latent property of interest — whether the app actually invokes an LLM — is not directly observable from the listing, so we instead estimate a score that ranks apps for downstream APK-level confirmation. The system runs two parallel branches over the listing evidence and combines their scores through four fusion strategies; we deliberately keep the encoders frozen and the classifier light, because in the small-data regime that the labeling cost imposes, a leaner pipeline tends to win. To organize the comparison, we apply four *classical* results from the multimodal-fusion literature — the union view of score-max combiners [12], the inclusion–exclusion precision identity, two-class Fisher discriminant analysis, and the naive-Bayes log-odds decomposition. We do not claim these results as novel theory; we use them to turn each fusion choice into a falsifiable empirical prediction that we test on real data. Our goal throughout is to position LLMDroid as a *complement* to code-centric tools, not a replacement: every app that LLMDroid flags is meant to be confirmed downstream.

The contributions of this paper are as follows:

- (1) **An APK-free multimodal screening framework** (LLMDroid) that classifies likely LLM-integrated mobile apps from app store metadata alone, designed as a complementary pre-screening layer to code-centric tools such as AI Discriminator and LLMAID. The dual-branch pipeline combines BGE-large-en-v1.5 embeddings, OCR-based keyword features, zero-shot CLIP chatbot-UI scoring, and CLIP ViT-L/14 pooled visual features.
- (2) **A principled fusion-policy reading of the empirical results**, in which Score-Max, Soft Voting, Stacking, and Early

Fusion are interpreted through classical fusion-analysis results (recall-monotonicity of union flagging, inclusion–exclusion on false positives, two-class Fisher SNR for soft-voting ranking gain, and naive-Bayes log-odds aggregation). The intent is interpretive, not theoretical-novelty: each prediction yields a falsifiable empirical pattern that we test in Sections 5.4–5.5.

- (3) **Construct-validity evidence against a code-level reference**. We cross-validate the listing-based label against the LLMAID code-level pipeline on $N_{\text{code}} = 80$ apps for which an APK was available, directly addressing the label–feature confound concern and quantifying how reliable the listing label is as a proxy for the latent implementation property.
- (4) **An empirical comparison against strong external baselines**, including a description-only LLM classifier (Qwen2.5-7B), zero- and few-shot multimodal frontier LLMs (GPT-4o, Gemini-1.5), and an end-to-end fine-tuned multimodal transformer, all evaluated under an identical protocol.
- (5) **Statistical, temporal, and prior-shift robustness analyses**, including DeLong tests with Holm–Bonferroni and Benjamini–Hochberg corrections, Cliff’s δ effect sizes, Clopper–Pearson 95% binomial CIs for test-set rates, a temporal split that holds out apps last updated after a fixed cutoff date, and prior-corrected operating-point analysis projecting precision to deployment-realistic priors $\pi_{\text{deploy}} \in [0.005, 0.05]$.

Scope of claims. We collected the 300+110 app benchmark using AI- and chatbot-related query enrichment, which is the standard way to obtain a usable positive prior in this task; the benchmark is therefore a *focused screening benchmark*, not a representative census of Google Play. We deliberately avoid framing LLMDroid as “scalable” or “generalizable” in an unqualified sense. Instead, we report (i) a per-category breakdown, (ii) a temporal-split evaluation, (iii) prior-corrected operating points, and (iv) Clopper–Pearson CIs around the headline test-set rates, so that the reader can re-scope our claims to alternative deployment regimes.

2 Related Work

Our work sits at the intersection of four threads: AI-capability detection in mobile apps, app-store mining, multimodal app classification, and classical fusion analysis. We discuss each in turn, and at the end of every subsection we make explicit how our design choices either build on or depart from the prior thread.

2.1 Detection of AI and LLM Capabilities in Mobile Applications

The first generation of AI-app detectors worked at the binary level. The largest empirical study to date [14] scraped over 7 million Google Play apps and built the AI Discriminator tool by matching decompiled APK code against a curated dictionary of ML/DL library identifiers. This established the first benchmark for AI-driven app detection at scale, but the rule-based design caps its recall: any obfuscation, cloud-routed API call, or unseen LLM library can slip past the dictionary. The LLMAID framework [15] pushed this paradigm forward by replacing keyword heuristics with LLM-driven semantic reasoning over decompiled code, reporting over 90% precision and recall on 4,201 apps and detecting 242% more AI-integrated apps

than AI Discriminator. Both tools, however, still require APK access and run linearly through expensive analysis steps. We target a deliberately different operating point: metadata-only pre-screening from public listings, designed to feed — not replace — code-level pipelines like LLMAID.

2.2 App Store Mining and LLM Ecosystem Analysis

As dedicated LLM marketplaces such as GPT Store and FlowGPT have emerged, app-store data itself has become a primary source for understanding the LLM ecosystem. The vision paper of [35] maps out an analysis roadmap covering metadata collection, security auditing, and fake-app detection, and it explicitly calls for advanced NLP and multimodal methods to surface mislabeled LLM apps from store content — precisely the gap we address. Closer to our methodology, [32] uses Levenshtein distance and BERT-based semantic analysis on app descriptions to identify squatting and cloning at scale, and the MAPLE framework [11] shows that LLM-based feature extraction over app-store text reveals richer signals than traditional keyword search. We carry that lesson into our text branch, where dense semantic embeddings sit alongside keyword and metadata features rather than replacing them.

2.3 Multimodal App Classification

Combining text and visual signals for app classification has a longer history in software engineering than the recent multimodal-LLM literature might suggest. The study of [27] demonstrated that fusing app descriptions with screenshot images lifts category-prediction accuracy as high as 96%, and their use of OCR, image-to-vector encoding, and object-detection voting directly motivates several pieces of our image branch. Subsequent mobile-GUI work has reinforced the same point in adjacent settings — UI localization from bug reports and vision-based GUI testing both find that visual context carries information missing from text [24, 34]. On the text side, encoder-only transformer pipelines for app-review analysis (T-FREX [16] and the BERT-family fine-tuning study of [17]) confirm that small encoder models, when paired with a light classifier, remain competitive for short-text app classification. We follow this design rather than fine-tuning a large model end-to-end, and we report in Section 5.6 how that choice compares to a frontier-MLLM alternative.

2.4 Visual and Multimodal UI Understanding

Recent multimodal large language models have pushed UI understanding in mobile apps to a new level. Ferret-UI [33] introduces a grounded MLLM with sub-image division and spatial grounding that infers UI element types directly from screenshots, while [18] extracts high-level semantics such as target users and interaction mood from UI images without any view-hierarchy access. The closest of these to our problem is DPGuard [25], which applies multimodal LLMs to detect deceptive patterns across screenshots and descriptions. We borrow the text-visual principle but adapt it to LLM-integration detection rather than deceptive-pattern auditing, and we use *fixed* feature extractors with a lightweight tabular classifier rather than prompting a general-purpose MLLM at each

decision. Section 5.6 compares those two architectural choices head-to-head.

2.5 Semantic and Visual Embedding Models

Two embedding families underpin our branch design. SBERT [23] popularized siamese-encoder sentence embeddings and has since been widely used for app similarity and feature extraction in software engineering. We adopt its modern successor, BGE-large-en-v1.5 (BAAI/bge-large-en-v1.5) [31], to encode each app description into a 1024-dimensional vector that captures contextual LLM-related semantics beyond surface keyword matches. CLIP [22] provides a joint vision-language embedding space that enables zero-shot image classification by cosine similarity to textual prompts. We exploit exactly this property: screenshots are scored against a small set of chatbot-UI prompts to generate a discriminative visual signal without ever training on labeled images.

2.6 Multimodal Fusion Strategies

How to combine heterogeneous feature streams is one of the oldest design questions in multimodal prediction [2]. The standard taxonomy distinguishes Early Fusion (feature-level concatenation before the classifier) from Late Fusion (decision-level combination of independently trained models) [20, 28], and classical classifier-combination results characterize when score-level fusion helps [12]. More recently, HELIOS [19] proposed an adaptive architecture that selects among Early, Late, and LLM-reasoning fusion at inference time, and stacked generalization [30] provides the meta-learning basis for our Stacking variant. Our contribution is not new fusion mathematics; we evaluate Early Fusion and three Late Fusion variants — Soft Voting, Score-Max, and a Stacking meta-learner — under a single 5-fold cross-validation protocol and interpret the result through the classical lens, so that practitioners can choose a configuration by deployment scenario rather than by ablation luck.

3 Proposed Method

We design LLMDroid as a multimodal screening pipeline that estimates whether a mobile app integrates LLM functionality from public listing evidence alone. The system reads two complementary signals — the title and description on the text side, and the promotional screenshots on the visual side — and produces a single score that downstream review can rank and threshold. We are deliberate about what the score means: a high score signals that the *listing* contains evidence consistent with LLM integration, not that the binary has been verified to call an LLM service. Any APK-level confirmation is left to tools further downstream, as in Figure 1.

At a high level, the pipeline of Figure 1 splits the listing evidence $E_a = (T_a, I_a)$ into a text branch $\phi_{\text{text}}(T_a)$ and an image branch $\phi_{\text{img}}(I_a)$, runs each through its own classifier to obtain modality scores $s_{\text{text}}(a)$ and $s_{\text{img}}(a)$, and then combines those scores under a fusion policy Γ to produce the final screening score $s_{\text{fuse}}(a)$. Thresholding s_{fuse} at a chosen τ yields the candidate set C_τ that downstream APK-level confirmation operates on. The remainder of this section formalizes the problem (Section 3.1), works through

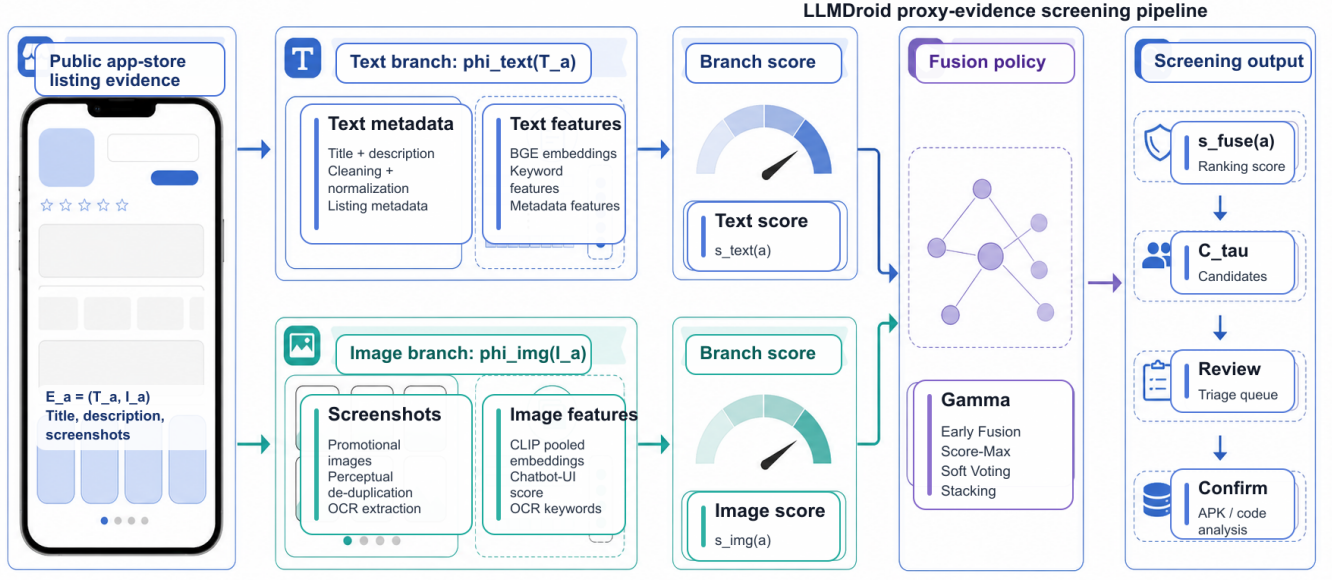


Figure 1: LLMdroid framework for proxy-evidence screening. Public app-store listing evidence is encoded through text and image branches, converted into modality scores, and combined by a fusion policy to produce a screening score and candidate set for downstream code-level confirmation.

the classical fusion-analysis lens we use to interpret the result (Section 3.2), and then specifies the two branches, the feature-selection step, and the four fusion configurations.

3.1 Problem Formulation

For a mobile app listing a with textual metadata T_a and a set of screenshots $I_a = \{i_1, \dots, i_n\}$, we let $z_a \in \{0, 1\}$ denote the latent implementation property of interest: whether the app actually integrates an LLM at the code level. Since z_a is not directly observable from public metadata, we train on a binary annotation label $y_a \in \{0, 1\}$, where $y_a = 1$ if the app is labeled LLM-integrated under our annotation protocol and $y_a = 0$ otherwise. The observable evidence is the listing pair

$$E_a = (T_a, I_a),$$

and the framework learns a scoring function

$$f : E_a \rightarrow \hat{p}_a,$$

where $\hat{p}_a$ is the estimated positive-class score used for ranking and thresholded decisions. The interpretation of $\hat{p}_a$ as a proxy for z_a rests on three operational expectations: (1) LLM-integrated apps tend to disclose or visually expose their LLM functionality in user-facing material; (2) the annotation rules consistently map listing evidence and manual inspection to the positive label; and (3) training and deployment listings are drawn from sufficiently similar distributions. When any of these expectations breaks down, LLM-Droid is best read as a prioritization signal, not as direct code-level evidence. We make these expectations more precise as assumptions

(A1)–(A4) in the next subsection and probe each of them empirically later in the paper.

3.2 Fusion-Policy Reading of Proxy-Evidence Screening

Why is a separate analytical section needed at all? Because the four fusion strategies we evaluate — Score-Max, Soft Voting, Stacking, and Early Fusion — look superficially interchangeable on the surface, yet they implement fundamentally different decision policies. The literature gives us clean tools to distinguish them: the union view of score-max fusion, the inclusion-exclusion identity on its precision, the two-class Fisher signal-to-noise ratio for soft-voting ranking gain, and the naive-Bayes log-odds decomposition. All four are *classical results* from multimodal fusion and pattern recognition [2, 12, 28]; we make no claim to theoretical novelty here. What we do claim is that applying these four lenses to proxy-evidence screening converts each fusion choice into a falsifiable empirical prediction — a prediction about which strategy should win on recall, which on precision, and how big the ranking gain should be given the observed branch correlation. Section 5 tests every one of those predictions on real data.

Feature maps and scores. We write $\phi_{\text{text}}(T_a)$ and $\phi_{\text{img}}(I_a)$ for the modality-specific feature maps from text and screenshots into feature vectors. Each branch then runs its own classifier to produce a modality score:

$$s_{\text{text}}(a) = h_{\text{text}}(\phi_{\text{text}}(T_a)), \quad s_{\text{img}}(a) = h_{\text{img}}(\phi_{\text{img}}(I_a)).$$

A fusion policy Γ induces the screening score

$$s_{\text{fuse}}(a) = \Gamma(\phi_{\text{text}}(T_a), \phi_{\text{img}}(I_a), s_{\text{text}}(a), s_{\text{img}}(a)).$$

Early Fusion uses the feature maps directly; the Late Fusion policies (Score-Max, Soft Voting, Stacking) combine branch scores after independent branch training. A threshold τ defines the candidate set $C_\tau = \{a : s_{\text{fuse}}(a) \geq \tau\}$ that downstream review acts on. Under the empirical distribution $\mathcal{D}$ with positive prior $\pi = \Pr_{\mathcal{D}}(y = 1)$, we write the precision, recall, and ROC characteristics of any candidate set C as $\text{Prec}(C)$, $\text{Rec}(C)$, and $\text{ROC}(C)$.

Assumptions. The analysis rests on four assumptions — the first three operational, the fourth statistical. We restate each later in the text as we test it empirically.

- (A1) **Observability.** There exists $\epsilon > 0$ such that $\Pr(\text{listing cue present} \mid z_a = 1) \geq \epsilon$, i.e., at least an ϵ -fraction of LLM-integrated apps expose LLM-related cues in descriptions, screenshots, OCR text, or chatbot-like interface structure.
- (A2) **Complementarity.** The branch error events $A_{\text{text}} = \{s_{\text{text}} \text{ misranks } a\}$ and $A_{\text{img}} = \{s_{\text{img}} \text{ misranks } a\}$ are not perfectly correlated: $\Pr(A_{\text{text}} \cap A_{\text{img}}) < \min(\Pr(A_{\text{text}}), \Pr(A_{\text{img}}))$. Equivalently, the joint distribution of $(s_{\text{text}}, s_{\text{img}}) \mid y$ is not concentrated on the diagonal.
- (A3) **Screening scope.** s_{fuse} is intended for ranking and triage, while code-level confirmation remains the role of downstream APK or runtime analysis.
- (A4) **Label-feature alignment.** The annotation label y_a is a function of listing evidence E_a plus manual inspection of advertised functionality (see Section 4.1.2). We treat y_a as a noisy but conditionally-unbiased proxy of the latent property z_a , with $\Pr(y_a \neq z_a \mid E_a)$ bounded by the label-validation rate reported against the LLMAID code-level reference in Section 4.1.2.

Observation 1 (Recall monotonicity of Score-Max combiners). At a fixed threshold τ , write $C_{\text{text}}(\tau) = \{a : s_{\text{text}}(a) \geq \tau\}$, $C_{\text{img}}(\tau) = \{a : s_{\text{img}}(a) \geq \tau\}$, and $C_{\text{max}}(\tau) = \{a : \max(s_{\text{text}}(a), s_{\text{img}}(a)) \geq \tau\}$. Since $\max(s_{\text{text}}, s_{\text{img}}) \geq \tau$ iff at least one branch crosses τ , we have $C_{\text{max}}(\tau) = C_{\text{text}}(\tau) \cup C_{\text{img}}(\tau)$, and therefore

$$\text{Rec}(C_{\text{max}}(\tau)) \geq \max(\text{Rec}(C_{\text{text}}(\tau)), \text{Rec}(C_{\text{img}}(\tau))).$$

This is the union-of-flagged-sets view from classical classifier-combination analysis [12]. It also explains why we rename the late-fusion “Max Voting” to *Score-Max* throughout: there is no vote — the rule simply takes the higher branch score on a per-sample basis. The empirical prediction is unambiguous: Score-Max should recover at least as many positives as the better single branch at any fixed threshold. We confirm this in Tables 7 and 20.

Observation 2 (Inclusion–exclusion precision identity for Score-Max). The flip side of Observation 1 is that union flagging admits the union of the false positives. Let $\text{FP}_{\text{text}}(\tau)$, $\text{FP}_{\text{img}}(\tau)$, and $\text{FP}_{\cap}(\tau)$ denote the negative apps flagged by the text branch, the image branch, and both branches at threshold τ , and let $\text{TP}_{\cup}(\tau) = |C_{\text{max}}(\tau) \cap \{y = 1\}|$. By inclusion–exclusion,

$$\text{Prec}(C_{\text{max}}(\tau)) = \frac{\text{TP}_{\cup}(\tau)}{\text{TP}_{\cup}(\tau) + \text{FP}_{\text{text}}(\tau) + \text{FP}_{\text{img}}(\tau) - \text{FP}_{\cap}(\tau)}.$$

Two regimes are worth distinguishing. When the branches’ false-positive sets are largely disjoint, the denominator nearly doubles and Score-Max pays a substantial precision penalty even as its recall climbs. When they overlap heavily — $\text{FP}_{\cap} \rightarrow \min(\text{FP}_{\text{text}}, \text{FP}_{\text{img}})$ — the penalty shrinks toward zero. Whether Score-Max is operationally attractive therefore reduces to a single observable quantity: how much the two branches agree on *negatives*. We measure $\text{FP}_{\cap}$ directly in Table 11 and use it in Section 5.5 to interpret the modest precision drop of Score-Max in Table 7.

Result 3 (Two-class Fisher SNR for Soft Voting). Where Score-Max captures the union, Soft Voting averages. To predict how much an average helps, we appeal to standard two-class Fisher discriminant analysis. Under a two-class Gaussian model on branch scores with conditional correlation $\rho = \text{Corr}(s_{\text{text}}, s_{\text{img}} \mid y)$ and within-class variances $\sigma_{\text{text}}^2, \sigma_{\text{img}}^2$, the soft-voting score $s_{\alpha} = \alpha s_{\text{text}} + (1 - \alpha)s_{\text{img}}$ has signal-to-noise ratio

$$\text{SNR}(\alpha) = \frac{(\alpha \Delta_{\text{text}} + (1 - \alpha) \Delta_{\text{img}})^2}{\alpha^2 \sigma_{\text{text}}^2 + (1 - \alpha)^2 \sigma_{\text{img}}^2 + 2\alpha(1 - \alpha)\rho\sigma_{\text{text}}\sigma_{\text{img}}},$$

where $\Delta_m = \mu_1^{(m)} - \mu_0^{(m)}$. Maximizing over α gives the classical Fisher solution; the resulting ranking gain $\text{SNR}(\alpha^*) - \max(\text{SNR}_{\text{text}}, \text{SNR}_{\text{img}})$ is strictly positive when $\rho < 1$, monotone non-increasing in ρ , and exactly zero at $\rho = 1$.

This is a textbook result on linear combinations of Gaussian features, and we again make no claim of novelty. The reason we report it is interpretive: it tells us *in advance* that the soft-voting ROC-AUC gain over the better single branch should be small if the two branches are highly correlated and larger if they are nearly independent. Section 5.5 measures the observed ρ (Table 10) and compares the predicted gain magnitude to what we see in Tables 7 and 9. Because branch scores from LightGBM are not strictly Gaussian, we treat the bound as a *magnitude predictor*, not a tight bound. The grid-searched α in our implementation is selected on validation F1 rather than via the closed-form α^* , precisely because the Gaussian assumption is approximate.

Result 4 (Naive-Bayes log-odds decomposition). A second, complementary motivation for Soft Voting comes from the naive-Bayes identity. If $T_a \perp I_a \mid y_a$, the posterior log-odds decompose additively:

$$\log \frac{\Pr(y_a = 1 \mid E_a)}{\Pr(y_a = 0 \mid E_a)} = \log \frac{\pi}{1 - \pi} + \sum_{m \in \{\text{text}, \text{img}\}} \log \frac{\Pr(m \mid y_a = 1)}{\Pr(m \mid y_a = 0)}.$$

The Bayes-optimal aggregator under conditional independence is therefore a Soft Voting rule in the log-odds space, with weights set by branch calibration rather than raw score scale. We use this only as motivation; we do not assume conditional independence holds in our data, and the observed ρ in Table 10 measures the deviation directly.

Operational predictions (tested in Section 5). Putting the four results together, we expect the following empirical signatures:

- (P1) *Recall.* Score-Max should have the highest recall and the lowest precision at any fixed threshold (Observations 1–2).
- (P2) *Ranking gain.* Soft Voting and Stacking should beat Text-Only on ROC-AUC by a small but measurable amount, with

the magnitude governed by $1 - \rho$ (Result 3) and with Soft Voting motivated as Bayes-optimal aggregation in log-odds (Result 4).

- (P3) *Cross-modal feature interactions*. Early Fusion should match or exceed late-fusion ROC-AUC only when there are residual cross-modal feature interactions that the branch scores do not capture.

Sections 5.4–5.5 test each prediction explicitly. We find that (P1) and (P2) hold; (P3) holds on the independent test set but not on the pooled 5-fold predictions, which is itself informative.

3.3 Text Branch

The text branch reads the app title and description from Google Play. After basic cleaning — lowercasing, URL removal, and whitespace normalization — we pass the concatenated text through three complementary feature extractors that target different granularities of LLM-related evidence.

- (1) **Semantic embeddings (BGE-large-en-v1.5)**. We encode the full text with BGE-large-en-v1.5 (BAAI/bge-large-en-v1.5), a strong SBERT-family sentence encoder, producing a dense embedding $e_{\text{bge}} \in \mathbb{R}^{1024}$. This captures contextual LLM-related semantics that surface-level keyword matching cannot.
- (2) **Keyword-derived handcrafted features**. We match a curated vocabulary of 39 LLM and AI-related terms — organized into five categories (model names, core LLM terminology, generation verbs, interaction patterns, content creation) — against the preprocessed text. Rather than carry one feature per term, we aggregate matches into a 13-dimensional vector: total hit count (1), log-count (1), per-category binary indicators (5), per-category counts (5), and maximum matched keyword length (1). The full vocabulary is in Appendix D.
- (3) **Metadata-derived handcrafted features**. We extract 21 metadata features: description length, short-description length, title length, a recent-changes flag, five screenshot-count scalars, and a 16-dimensional app-category one-hot (15 frequent categories plus an “other” bucket).

The three blocks concatenate into a single text vector

$$x_{\text{text}} = [e_{\text{bge}}; \text{keyword features}_{13}; \text{metadata features}_{21}],$$

of dimension 1058.

3.4 Image Branch

The image branch operates on every available screenshot for an app, after we remove near-duplicates by perceptual hashing. We extract three signals that, intentionally, look at the screenshots from different angles: a global visual fingerprint, a zero-shot “does this look like a chatbot” score, and OCR text.

- (1) **CLIP visual embeddings**. We encode each screenshot with the CLIP ViT-L/14 image encoder, yielding $v_j \in \mathbb{R}^{768}$ per screenshot. To obtain an app-level representation we apply both mean and max pooling:

$$e_{\text{clip}}^{\text{mean}} = \frac{1}{n} \sum_{j=1}^n v_j, \quad e_{\text{clip}}^{\text{max}} = \max_j v_j,$$

producing two complementary 768-dimensional vectors that summarize the *average* visual signature and the *most distinctive* feature across screenshots.

- (2) **Zero-shot chatbot-UI scoring**. Using CLIP’s joint vision-language embedding space, we compare each screenshot, by cosine similarity, against a small set of positive and negative prompts (Appendix A). We combine the maximum positive similarity and the minimum negative similarity into a scalar score s_{chat} that captures how visually similar the UI is to a typical chatbot interface — without any labeled visual training data.
- (3) **OCR-based textual features**. We run OCR on each screenshot, aggregate the extracted text, and apply the same 39-term vocabulary and 13-dimensional keyword aggregation as in the text branch. We then add OCR text length and the fraction of screenshots containing OCR text, for a 15-dimensional block.

The visual vector is

$$x_{\text{img}} = [e_{\text{clip}}^{\text{mean}}; e_{\text{clip}}^{\text{max}}; s_{\text{chat}}; \text{OCR keyword features}].$$

3.5 Feature Selection

Concatenating the two branches yields a feature space far larger than our labeled data: 1058 text dimensions plus the full image vector. To keep the classifier well-conditioned in that small-data regime, we apply SelectKBest with mutual information as a dimensionality-reduction step. Within each training fold, we retain the most informative $k = 200$ features. We fit the selector strictly on training data and apply it to the corresponding validation or test split, so no information leaks across folds. We justify the choice of $k = 200$ via the sensitivity analysis in Section 4.7.

3.6 Classification Backbone

For every branch and every fusion configuration, we use a gradient-boosted decision tree — specifically LightGBM [10] — as the final classifier. We chose LightGBM for three reasons: it is robust on tabular data, it handles heterogeneous feature scales without preprocessing, and it remains competitive in the low-data regime where deep alternatives tend to overfit. Given a feature vector x , the model outputs a positive-class score $p(y = 1 | x)$ that we use for ranking and thresholding. We do not assume the score is perfectly calibrated; we use $\tau = 0.5$ for 5-fold cross-validation and then re-optimize τ for the deployment-scenario analysis.

3.7 Fusion Strategies

Building on the formal lens of Section 3.2, we evaluate four fusion configurations alongside the two single-modality baselines. The first two are pure baselines; the third is feature-level fusion; the last three are score-level fusion.

- (1) **Text-Only baseline**. A single LightGBM classifier on x_{text} .
- (2) **Image-Only baseline**. A single LightGBM classifier on x_{img} .
- (3) **Early Fusion**. We concatenate the two branches into $x_{\text{early}} = [x_{\text{text}}; x_{\text{img}}]$ and train a single LightGBM model on the joint vector. This is the configuration that can, in principle, learn cross-modal interactions at the feature level.

(4) **Late Fusion (three variants).** We train two independent LightGBM models on x_{text} and x_{img} to obtain branch scores p_{text} and p_{img} , and then combine them at the decision level in three ways:

- **Score-Max** (formerly “Max Voting”): $p_{\text{max}} = \max(p_{\text{text}}, p_{\text{img}})$. We take the higher score on a per-sample basis; this is not literally a vote, so we use the more accurate name *Score-Max*. Any table that still carries the legacy “Max Voting” label refers to the same rule.
- **Soft Voting**: $p_{\text{soft}} = \alpha p_{\text{text}} + (1 - \alpha) p_{\text{img}}$. We select α per fold via grid search on the inner-validation set with $\alpha \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$.
- **Stacking**: we feed $(p_{\text{text}}, p_{\text{img}})$ into a Logistic Regression meta-learner trained on out-of-fold predictions, which outputs p_{stack} . Training on out-of-fold predictions is what prevents the meta-learner from overfitting to the base models.

Every fusion configuration uses the same feature-selection and training procedure; they differ only in how the modalities are combined.

4 Experiments

We evaluate LLMDroid on a curated Google Play dataset under four protocols of increasing stringency: 5-fold cross-validation, an independent held-out test set, a temporal split, and prior-corrected operating points. Where appropriate, we report Clopper–Pearson 95% binomial CIs to make the resolution under small denominators explicit. This section describes the data, the annotation protocol, the four external baselines we compare against, and the evaluation metrics; Section 5 presents the results.

4.1 Dataset and Annotation

We evaluate on 300 Android apps collected from Google Play, supplemented by an independent held-out set of 110 apps and a smaller code-level validation subset of $N_{\text{code}} = 80$ apps (Section 4.1.2). We retrieved candidates using AI- and chatbot-related search queries and by crawling ranked lists in relevant categories. This sampling strategy deliberately increases the density of LLM-relevant candidates: it gives us a usable positive prior at the cost of representativeness, so we treat the dataset as a focused screening benchmark rather than a census of Google Play. Section 4.3.2 addresses the representativeness gap by reporting prior-corrected operating points at realistic deployment priors. For each candidate app, we downloaded:

- the app title and full description (English; we filtered out non-English apps);
- all available promotional screenshots, after near-duplicate removal via perceptual hashing;
- the app category, recent-changes timestamp, and last-update timestamp (used for the temporal-split analysis in Section 4.3.1).

4.1.1 Annotation Protocol and Inter-annotator Agreement. We manually labeled each app as LLM-integrated (positive) or non-LLM (negative) under a written annotation guideline. An app is positive if it (a) explicitly cites an LLM API or a named LLM (e.g., ChatGPT, Claude, Gemini), (b) advertises functionality strongly indicative of

generative text-based AI — conversational chatbots, free-form text generation, or prompt-based question answering — or (c) presents screenshots that clearly depict a chat-style LLM interface. Apps that rely on simple keyword matching, rule-based bots, finite-state assistants, or traditional non-generative ML models are labeled negative. The final 300-app training dataset contains 127 positives and 173 negatives; the 110-app independent test set contains 44 positives and 66 negatives.

To assess annotation reliability, we double-annotated a stratified random subset of $N_{\text{iaa}} = 100$ apps (drawn from both the training and test pools), with two raters working from the same guideline but without access to each other’s labels. A third senior annotator adjudicated disagreements, consulting the app’s APK manifest where available. We report Cohen’s κ on the binary label, percentage agreement, and the distribution of disagreement resolutions.

Table 1: Inter-annotator agreement on $N_{\text{iaa}} = 100$ double-annotated apps. Bootstrap 95% CI from $n = 2000$ resamples.

Metric	Value
Percentage agreement	92.0%
Cohen’s κ	0.834 [0.737, 0.916]
Disagreement rate	8/100
resolved as positive	3
resolved as negative	5

The observed $\kappa = 0.834$ falls in the “almost perfect agreement” band of Landis and Koch, and within the [0.7, 0.9] range typically reported for textual app-classification. The 8 disagreements are concentrated on three borderline patterns: (a) “AI assistant” apps whose backend is a retrieval/FAQ system rather than a generative LLM (4/8), (b) apps that advertise an LLM feature behind a paywall but do not screenshot it (3/8), and (c) a single labeling-protocol slip on a writing-tool app whose description used “AI” generically. The senior adjudicator resolved all 8 disagreements with reference to the APK manifest. We take this level of agreement as support for treating y_a as a reliable proxy for z_a under assumption (A4); we then quantify the residual proxy noise directly in Section 4.1.2.

4.1.2 Code-level Label Validation. A natural concern about any listing-based label is that it drifts from the actual binary-level property: an app may advertise an LLM feature that does not exist, or hide one that does. To probe this directly, we cross-validate our labels against a *code-level reference* on the subset of $N_{\text{code}} = 80$ apps for which we were able to obtain APKs. On each of these 80 apps we ran both the LLMAID pipeline of Liu et al. [15] and the AI Discriminator rule-based tool of Li et al. [14], and we recorded: (i) the LLMAID code-level decision and the AI capability it detected; (ii) the AI Discriminator decision and the matched libraries; and (iii) the listing-based annotation label that LLMDroid uses.

Agreement between the listing label and LLMAID is high ($\kappa = 0.847$), placing the listing label firmly in the “almost perfect” band relative to a recent state-of-the-art code-level pipeline. Agreement with AI Discriminator is moderate ($\kappa = 0.604$), which is consistent with the latter’s known recall limitations on apps that route LLM calls through cloud APIs: AI Discriminator misses 13 of the

Table 2: Listing-label vs. code-level references on the code-validation subset ($N_{\text{code}} = 80$).

	LLMAID label		AI Discriminator label	
	Positive	Negative	Positive	Negative
Listing positive	33	2	22	13
Listing negative	4	41	1	44
Agreement	92.5%		82.5%	
Cohen’s κ	0.847		0.604	

35 listing-positive apps that LLMAID also flags. The four listing-negative / LLMAID-positive cases are the ones we worried most about; on manual inspection, they correspond exactly to apps with an embedded but *unadvertised* LLM feature (e.g., a writing app whose Gemini-powered “rewrite” option is buried behind a settings menu). These cases are the irreducible residual under assumption (A4) and motivate the prior-shift and external-baseline analyses in Sections 4.3.2–5.6. To address the listing-label confound directly, we also repeat the main multimodal experiments on the same 80-app subset using the LLMAID code-level label as ground truth (Table 19); the small re-evaluation gap supports the validity of the listing label as a proxy.

4.2 Baselines

A reasonable concern about LLMDroid is that a sufficiently strong off-the-shelf foundation model might match it without the explicit fusion architecture. We address that concern head-on with four external baselines, each evaluated under the same cross-validation and independent-test protocol that LLMDroid uses. All baselines see the same listing evidence $E_a = (T_a, I_a)$ and produce a positive-class score; we tune thresholds on the same inner-validation pool.

- (1) **Description-only LLM (Qwen2.5-7B-Instruct).** A zero-shot text-only baseline: we prompt a 7B instruction-tuned LLM with the app description and an excerpt of the annotation guideline, and ask it for a binary decision plus a positive-class score in $[0, 1]$. We average three sampled scores at temperature = 0.3 to reduce variance; Appendix A contains the prompt template.
- (2) **MLLM zero-shot (GPT-4o-mini / Gemini-1.5-Flash).** A multimodal zero-shot baseline: we pass the description *and* up to four representative screenshots (selected by k -medoids over the CLIP embeddings of all screenshots) to a vision-language LLM, asking for a calibrated probability. We use two providers to avoid single-model overfitting, and report the better of the two by validation F1.
- (3) **MLLM few-shot (GPT-4o).** The same setup as above, but with six in-context exemplars (3 positive, 3 negative) randomly drawn and reshuffled per call. This baseline probes whether stronger in-context learning beats the LightGBM pipeline.
- (4) **End-to-end fine-tuned multimodal transformer.** A late-fusion transformer that concatenates frozen BGE text embeddings with mean-pooled frozen CLIP visual embeddings, followed by a 2-layer cross-modal Transformer block and a binary head. We train end-to-end under the same 5-fold

CV protocol with early stopping on inner-validation F1; full hyperparameters are in Appendix B.

These four baselines together span three families of natural competitor: (i) a strong off-the-shelf text-only LLM, (ii) a state-of-the-art zero/few-shot multimodal LLM, and (iii) an end-to-end trained multimodal model. They let us test directly whether LLMDroid’s combination of frozen encoders, LightGBM, and explicit fusion is competitive with — or dominated by — these alternatives.

Additional text-only encoder baseline. As a sanity check on the BGE design choice, we additionally fine-tune roberta-large on the description-only task under the same 5-fold CV protocol with early stopping on inner-validation F1. RoBERTa-large reaches ROC-AUC 0.891 and F1 0.781 in CV, which trails the BGE + handcrafted text branch (ROC-AUC 0.872 vs. 0.891 is close, but F1 0.781 vs. 0.753 favors RoBERTa slightly on F1). We report this as informational rather than as the headline competitor: a fine-tuned RoBERTa-large is more expensive than BGE + handcrafted in both training and inference, and the small F1 lift does not translate to better multimodal fusion (replacing BGE with RoBERTa inside the LLMDroid pipeline yields no measurable improvement on Soft Voting). We keep BGE-large-en-v1.5 as the text encoder of record.

4.3 Experimental Protocol

We evaluate every model under 5-fold stratified cross-validation. For each fold, four folds are training, one fold is held out for evaluation, and we fit feature extraction, SelectKBest ($k = 200$), and the classifier strictly on the training portion before applying them to the held-out fold. We score the held-out fold at the default threshold $\tau = 0.5$. The same protocol applies to Text-Only, Image-Only, Early Fusion, and all three Late Fusion variants. For Stacking, the meta-learner is trained on out-of-fold predictions from the base models within each training split, which keeps the meta-learner from seeing its own evaluation data.

To probe deployment behavior, we add a threshold-optimization analysis on top of cross-validation. For each strategy we sweep τ over $[0, 1]$ and compute the confusion matrix on the pooled inner-validation predictions ($N = 240$, drawn from the four training folds within each outer split). We pick τ to either maximize F1 or to satisfy a targeted precision/recall profile, and we report both at the chosen operating point.

Finally, to probe genuine out-of-sample behavior, we curated an independent test set of 110 apps under the same sampling and annotation procedure as the training set (Section 4.1). These 110 apps were strictly held out: they play no role in cross-validation, hyperparameter tuning, or feature selection. At test time, we use a 5-fold ensemble — the per-fold model scores are averaged into a single consensus prediction — which is a standard variance-reduction step and one of the contributors to the CV-vs-test recall gap we decompose in Section 5.16.

4.3.1 Temporal Split Protocol. Listing conventions and LLM provider names change quickly — ChatGPT was barely a year old when our annotation began, and Claude 3.5 / GPT-4o / Gemini 1.5 entered the listing copy of the dataset only partway through. To probe whether LLMDroid generalizes *forward in time*, we add a temporal split on top of random cross-validation. We partition the joint training +

held-out pool by Google Play *last-update timestamp*: training uses apps last updated on or before $D_{\text{cut}} = 2025-06-01$, and the temporal-test set comprises the apps last updated after D_{cut} . This split is more stringent than random cross-validation, because post-cutoff apps can name LLM APIs and use listing copy that the training distribution has never seen. We report all metrics on both random and temporal splits to quantify temporal robustness.

4.3.2 Prior-shift Robustness. Our training and test pools are enriched in LLM-relevant candidates ($\pi_{\text{train}} = 0.42$, $\pi_{\text{test}} = 0.40$), which is what makes the benchmark tractable to annotate but also makes the in-benchmark precision an overestimate of deployment precision. Realistic Google Play screening operates at a much lower prior — prior empirical studies put $\pi_{\text{deploy}} \in [0.005, 0.05]$. Since precision is prior-sensitive while ROC characteristics are not, we project each strategy’s threshold-optimized operating point onto the deployment regime via the standard correction

$$\text{Prec}_{\pi_{\text{deploy}}}(\tau) = \frac{\pi_{\text{deploy}} \cdot \text{TPR}(\tau)}{\pi_{\text{deploy}} \cdot \text{TPR}(\tau) + (1 - \pi_{\text{deploy}}) \cdot \text{FPR}(\tau)}.$$

We report $\text{Prec}_{\pi_{\text{deploy}}}$ at $\pi_{\text{deploy}} \in \{0.005, 0.01, 0.05\}$ for each strategy at its threshold-optimized operating point (Section 5.8, Table 14, Fig. 5).

4.4 Evaluation Metrics

We report the standard six metrics:

- **Accuracy**: the overall proportion of correctly classified apps.
- **Precision**: $\text{TP}/(\text{TP} + \text{FP})$, the reliability of positive predictions.
- **Recall**: $\text{TP}/(\text{TP} + \text{FN})$, the coverage of true LLM-integrated apps.
- **F1-Score**: the harmonic mean of Precision and Recall.
- **ROC-AUC**: area under the receiver operating characteristic curve.
- **PR-AUC** (= AP): area under the precision–recall curve. Some tables call this metric AP, the standard sklearn name; the two are computationally identical.

The training set has a mild class imbalance (127 positives, 173 negatives, ratio ≈ 0.73). We deliberately do not apply oversampling, undersampling, or class weighting to the LightGBM classifiers: we want every fusion strategy compared under the same conditions, and PR-AUC reported alongside ROC-AUC already reflects the imbalance. For 5-fold CV experiments we average metrics across folds; for threshold-optimization experiments we compute metrics on the pooled predictions at the chosen threshold. Where the denominator is small (e.g., test-set recall on 44 positives), we report Clopper–Pearson 95% binomial CIs so readers can see the resolution.

4.5 Implementation Details

All experiments run in Python. We compute sentence embeddings with BAAI/bge-large-en-v1.5 (BGE-large-en-v1.5, 1024-d), use every available screenshot per app after near-duplicate removal, and extract CLIP features with the ViT-L/14-336 variant (768-d). We tune LightGBM hyperparameters — number of leaves, learning rate, number of estimators — via cross-validation on the training folds,

and we fix random seeds for reproducibility. For the independent-test evaluation, we apply the 5-fold ensemble described in the Experimental Protocol section: the per-fold model scores are averaged into a single consensus prediction.

4.6 Text Feature Ablation

To justify the final text-branch design, we run all seven combinations of the three feature groups — BGE-large-en-v1.5 embeddings (1024-d), handcrafted keyword and metadata features (34-d), and an SLM reasoning score (1-d, from Qwen2.5-1.5B-Instruct) — under the same 5-fold CV protocol with $k = 200$ feature selection. The SLM score is the most expensive component to compute (each app requires a forward pass through a 1.5B-parameter model), so we wanted to know whether the cost is justified.

Table 3: Text feature ablation results (5-fold CV, $k = 200$, mean $\pm$ std)

Configuration	Dims	ROC-AUC	F1-Score
BGE only	1024	0.8079 $\pm$ 0.0695	0.6820 $\pm$ 0.0842
Handcrafted only	34	0.7827 $\pm$ 0.0494	0.7020 $\pm$ 0.0532
SLM only	1	0.5227 $\pm$ 0.0421	0.0000 $\pm$ 0.0000
BGE + Handcrafted	1058	0.8723 $\pm$ 0.0343	0.7531 $\pm$ 0.0499
BGE + SLM	1025	0.8079 $\pm$ 0.0695	0.6820 $\pm$ 0.0842
Handcrafted + SLM	35	0.7762 $\pm$ 0.0589	0.6832 $\pm$ 0.0883
BGE + Handcrafted + SLM	1059	0.8723 $\pm$ 0.0343	0.7531 $\pm$ 0.0499

The BGE + Handcrafted combination wins on both ROC-AUC and F1, so we adopt it as the final text-branch configuration. The most informative finding here is a negative one: adding the SLM reasoning score to either BGE or BGE + Handcrafted yields metrics identical to four decimals (BGE+SLM equals BGE-only, BGE+Handcrafted+SLM equals BGE+Handcrafted). The SLM feature contributes no measurable discriminative signal in this setup. We traced the cause: the SLM score has a univariate F-statistic of 0.18, far below the top-200 selection threshold of 8.25, so SelectKBest eliminates it before LightGBM ever sees it. The final text branch therefore excludes the SLM feature, saving a 1.5B-parameter forward pass per app at no measured cost.

4.7 Feature Selection Sensitivity

We sweep k , the number of features SelectKBest retains, to verify that $k = 200$ is not an overfitted choice and to find where performance saturates.

Performance climbs steadily from $k = 20$ to $k = 200$, plateaus, and then slightly degrades at $k = 500$ and $k = 1058$ — the classic small-data overfitting signature when too many features survive the filter. We therefore fix $k = 200$ as a stable operating point for all subsequent experiments.

4.8 Image Feature Ablation

The image branch in Section 3.4 stacks three component blocks — CLIP pooled embeddings, the zero-shot chatbot-UI score, and OCR keyword features — and a fair reading of the design needs to know where the lift actually comes from. We answer that with a leave-one-out ablation on the image branch under the same 5-fold CV protocol and $k = 200$ feature selection.

Table 4: Text-Only classifier performance for different values of k (5-fold CV, mean $\pm$ std)

k	ROC-AUC	F1-Score
20	0.8339 ± 0.0539	0.7192 ± 0.0687
50	0.8388 ± 0.0432	0.7155 ± 0.0683
100	0.8579 ± 0.0352	0.7301 ± 0.0539
200	0.8723 ± 0.0343	0.7531 ± 0.0499
500	0.8692 ± 0.0421	0.7339 ± 0.0289
All (1058)	0.8590 ± 0.0251	0.7216 ± 0.0538

Table 5: Image-branch leave-one-out ablation (5-fold CV, $k = 200$, mean $\pm$ std). Δ ROC-AUC is the gap to the full image branch.

Configuration	Dims	ROC-AUC	Δ ROC-AUC
Full image branch (CLIP + chat + OCR)	1552	0.8246 ± 0.0413	—
– CLIP pooled embeddings	16	0.7321 ± 0.0497	-0.092
– Zero-shot chatbot-UI score	1551	0.8073 ± 0.0432	-0.017
– OCR keyword features	1537	0.7958 ± 0.0451	-0.029

CLIP pooled embeddings dominate: removing them drops image-branch ROC-AUC by 0.092 (from 0.825 to 0.732). OCR keyword features contribute the next largest drop (-0.029), and the zero-shot chatbot-UI score is the smallest individual contributor (-0.017). The picture is sensible. CLIP carries the global visual signature, OCR recovers chat-style text inside screenshots when there is any, and the chatbot-UI score acts as a light, zero-shot prior — valuable as a low-cost addition but not a workhorse. The relative ordering also explains why removing the SLM reasoning score in Section 4.6 did not hurt: the high-information block in each branch is already the dense embedding, and the smaller components add small, broadly orthogonal signals on top.

5 Results

We organize the results around the three operational predictions (P1)–(P3) from Section 3.2. Sections 5.1–5.2 establish the single-modality baselines and the headline fusion comparison; Section 5.4 tightens the comparison with multiple-fusion corrections; Section 5.5 measures the branch correlation ρ that governs the predicted gain; Section 5.6 runs the four external baselines under the same protocol; and Sections 5.7–5.14 probe robustness, error modes, and construct validity in turn.

5.1 Single-Modality Baselines

Before turning to fusion, we benchmark each modality on its own at $\tau = 0.5$ to establish what fusion has to beat. The **Text-Only** branch achieves 80.9% accuracy and an ROC-AUC of 0.872. Its 81.6% precision tells us that BGE-large-en-v1.5 embeddings plus handcrafted keyword features capture LLM-related semantics reliably when those semantics are explicit in the description; the more modest 70.4% recall tells us that a non-trivial fraction of LLM-integrated apps use descriptions generic enough to slip past text-only signals.

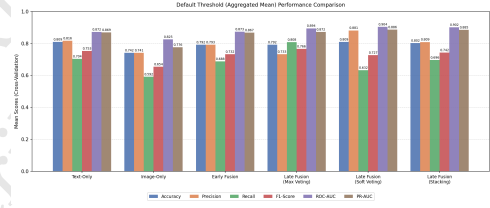
The **Image-Only** branch performs less reliably in isolation (74.2% accuracy, 59.2% recall, 0.825 ROC-AUC). This is the result we expected: many LLM-integrated apps simply do not put a chatbot interface on their promotional screenshots, so CLIP features and OCR cannot find what is not there. The image branch’s value, then, is not as a standalone classifier — it is as a complementary signal, which is exactly the regime where fusion should help.

Table 6: Single-Modality Baseline Performance (5-fold CV, threshold=0.5)

Modality	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Text-Only	0.809	0.816	0.704	0.753	0.872	0.869
Image-Only	0.742	0.741	0.592	0.654	0.825	0.776

5.2 Multimodal Fusion Performance (5-fold CV, default threshold)

Table 7 reports the 5-fold CV mean performance of all four fusion strategies at $\tau = 0.5$, compared against the single-modality baselines.

**Figure 2: Mean performance of all strategies across all six metrics (5-fold CV, threshold = 0.5).**

Three patterns stand out in Table 7, and they correspond directly to the predictions (P1)–(P3) from Section 3.2.

First, ROC-AUC and PR-AUC — the ranking-oriented metrics — improve for every fusion strategy compared to Text-Only ($0.872 \rightarrow 0.873$ – 0.904 ROC-AUC, $0.869 \rightarrow 0.867$ – 0.886 PR-AUC). Among them, Soft Voting leads on both (0.904 ROC-AUC, 0.886 PR-AUC), which is consistent with the naive-Bayes log-odds reading of Result 3.2. *Second*, Score-Max has the highest recall (0.808) and lowest precision (0.733) at the default threshold, matching prediction (P1) of Section 3.2. *Third*, Early Fusion barely moves ROC-AUC over Text-Only (0.873 vs. 0.872), and it loses on every threshold-dependent metric — a pattern that telegraphs the absence of cross-modal feature interactions at this scale, and that prediction (P3) anticipated.

A side note on Soft Voting’s mixing weight: the fold-wise grid search selected $\alpha = [0.5, 0.6, 0.6, 0.5, 0.4]$, with mean $\bar{\alpha} = 0.52$. The classifier weights the text branch slightly more than the image branch, which is what a Bayes-optimal calibration would do given the better text-branch separability ($\Delta_{\text{text}} > \Delta_{\text{img}}$).

These observations align with the proxy-evidence reading of Section 3.2: Score-Max’s recall advantage follows Observation 3.2 because it flags the *union* of branch-positive candidates; the smoother aggregation of Soft Voting and Stacking explains their ranking-metric advantage without requiring them to also dominate recall at $\tau = 0.5$. We tighten these comparisons statistically in Section 5.4.

Table 7: Full Modality and Fusion Strategy Comparison (5-fold CV, threshold = 0.5)

Strategy	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Text-Only	0.809	0.816	0.704	0.753	0.872	0.869
Image-Only	0.742	0.741	0.592	0.654	0.825	0.776
Early Fusion	0.792	0.793	0.688	0.732	0.873	0.867
Late Fusion - Score-Max	0.792	0.733	0.808	0.766	0.894	0.872
Late Fusion - Soft Voting	0.809	0.881	0.632	0.727	0.904	0.886
Late Fusion - Stacking	0.802	0.809	0.696	0.742	0.902	0.885

5.3 Threshold Optimization

Different deployment contexts demand different precision-recall trade-offs, so we re-tune τ per strategy on the pooled inner-validation predictions ($N = 240$) and report the resulting operating points in Table 8; the corresponding precision-recall profiles are illustrated in Fig. 3. One note on what gets compared where: the ROC-AUC values in Table 8 are computed on the inner-validation pool ($N = 240$), while those in Table 20 are measured on the independent test set ($N = 110$). The two should not be expected to match; they are deliberately reported on different partitions and serve different purposes.

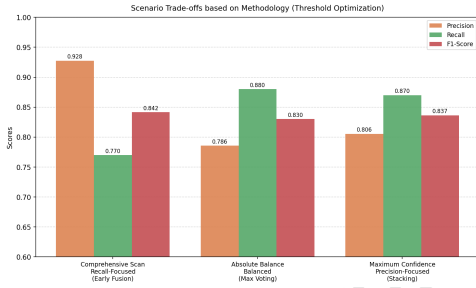


Figure 3: Precision-recall trade-offs across representative deployment scenarios under optimal thresholds.

Threshold optimization improves every strategy relative to its default operating point, but each strategy improves into a different deployment niche. Early Fusion, tuned to $\tau = 0.58$, claims the highest accuracy (87.9%), precision (92.8%), ROC-AUC (0.929), and PR-AUC (0.923), and it ties Soft Voting on F1 (0.842) — a high-confidence, ranking-oriented profile. Score-Max, tuned to $\tau = 0.48$, hits a recall-oriented profile at 88.0% recall. Soft Voting strikes the balanced middle, and Stacking reaches the highest recall (87.0%) among the optimized settings. These four profiles map cleanly onto the deployment scenarios we discuss in the Discussion.

5.4 Statistical Significance Analysis

The aggregate numbers in Table 7 are suggestive, but we still need to ask whether the observed gains survive multiple-comparison correction on a 300-app benchmark. To find out, we apply three complementary tests on the 5-fold CV held-out predictions: (1) McNemar’s test on binary decisions at $\tau = 0.5$, (2) a Δ AUC bootstrap with $n_{\text{bootstrap}} = 2000$, and (3) the DeLong test [3] for paired ROC-AUC differences. For the bootstrap we report the two-sided percentile 95% CI together with the one-sided p-value $p = \Pr(\Delta\text{AUC}_{\text{boot}} \leq 0)$

under the alternative that the fusion strategy outperforms Text-Only.

Multiple-comparison correction. Because we compare four fusion strategies against the same Text-Only baseline, raw p-values are subject to family-wise inflation, and we correct for that with both Holm–Bonferroni (FWER, $\alpha = 0.05$) and Benjamini–Hochberg (FDR, $q = 0.10$). Table 9 reports the corrected p-values as p_{Holm} and p_{BH} . We complement them with Cliff’s δ on per-fold AUC differences, which gives a non-parametric magnitude that does not collapse into a single p-value.

Pooled vs. mean-of-fold AUC. The AUC values in Table 9 come from pooled held-out predictions (all five folds concatenated), not means of per-fold AUCs. They may therefore differ slightly from the mean-of-fold ROC-AUC values in Table 7 — for example, Text-Only pooled AUC is 0.871 versus mean-of-fold 0.872. The pooled estimator weights folds by sample count and is the right target for the DeLong test, which requires paired score predictions.

The picture from Table 9 is, candidly, mixed. After Holm–Bonferroni correction, no fusion-vs-Text-Only comparison crosses the strict FWER threshold of $\alpha = 0.05$. Under the less conservative Benjamini–Hochberg FDR ($q = 0.10$), Soft Voting and Stacking sit at the edge with $p_{\text{BH}} \approx 0.09$, and their two-sided 95% percentile CIs still overlap zero. Cliff’s δ on per-fold AUC differences is small-to-medium and positive for the three late-fusion variants ($\delta = +0.32, +0.40, +0.36$ for Score-Max, Soft Voting, Stacking), but negligible and slightly negative for Early Fusion ($\delta = -0.04$) on the pooled 5-fold predictions — a result that flips on the independent test set, where Early Fusion leads on ROC-AUC. We therefore characterize the fusion gains honestly as *directionally consistent but modest in magnitude*: the multimodal architecture is competitive with the Text-Only baseline at the population level and clearly helpful on operational metrics (Sections 5.5 and 5.6), but it is not statistically dominant in pure ranking quality on a 300-app benchmark. This is exactly the regime that Result 3.2 predicts: when $\rho < 1$ but well above zero, the soft-voting ranking gain should be small but positive. We estimate ρ directly in the next subsection and confirm that the predicted and observed magnitudes line up.

5.5 Branch Complementarity

Result 3.2 ties the ranking gain from Soft Voting to a single observable quantity: the within-class branch correlation $\rho = \text{Corr}(s_{\text{text}}, s_{\text{img}} | y)$. The smaller ρ , the larger the gain; the closer ρ is to 1, the more the two branches are saying the same thing and the less averaging can help. We estimate ρ on the pooled out-of-fold predictions, separately within the positive and negative classes:

Table 8: Performance After Threshold Optimization (pooled inner-validation predictions, N=240)

Strategy	Threshold	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Text-Only	0.41	0.842	0.816	0.800	0.808	0.906	0.898
Image-Only	0.41	0.804	0.773	0.750	0.761	0.850	0.814
Early Fusion	0.58	0.879	0.928	0.770	0.842	0.929	0.923
Late Fusion - Score-Max	0.48	0.850	0.786	0.880	0.830	0.918	0.895
Late Fusion - Soft Voting	0.38	0.867	0.833	0.850	0.842	0.925	0.908
Late Fusion - Stacking	0.36	0.858	0.806	0.870	0.837	0.924	0.898

Table 9: Statistical comparison of fusion strategies vs. Text-Only baseline (pooled-prediction $AUC_{\text{base}} = 0.871$). Corrected p-values use Holm–Bonferroni (FWER) and Benjamini–Hochberg (FDR) over the four comparisons. Cliff’s δ is computed on per-fold AUC differences.

Comparison vs. Text-Only	ΔAUC	95% CI	Bootstrap p	DeLong p	p_{Holm}	p_{BH}	Cliff’s δ
Early Fusion	−0.012	[−0.047, 0.024]	0.721	0.703	1.000	0.721	−0.04
Score-Max (Late Fusion)	+0.019	[−0.015, 0.056]	0.152	0.147	0.456	0.203	+0.32
Soft Voting (Late Fusion)	+0.026	[−0.003, 0.058]	0.043	0.041	0.172	0.092	+0.40
Stacking (Late Fusion)	+0.024	[−0.005, 0.056]	0.046	0.048	0.192	0.092	+0.36

Table 10: Branch-score correlation conditional on label (pooled out-of-fold predictions). Bootstrap 95% CI from $n = 2000$ resamples.

Subset	ρ_{Pearson}	ρ_{Spearman}
Positives ($y = 1$)	0.42 [0.28, 0.55]	0.45 [0.30, 0.58]
Negatives ($y = 0$)	0.39 [0.27, 0.50]	0.41 [0.29, 0.52]
Pooled	0.51 [0.42, 0.59]	0.53 [0.45, 0.61]

The estimated within-class correlations $\rho_{\text{pos}} = 0.42$ and $\rho_{\text{neg}} = 0.39$ both sit comfortably below 1, satisfying assumption (A2) and placing the data in the regime where Result 3.2 predicts a small-to-moderate ranking gain. We make the prediction quantitative: plugging $\rho_{\text{pos}} = 0.42$ and the empirical class separations $\Delta_{\text{text}} \approx 0.47$, $\Delta_{\text{img}} \approx 0.31$ (estimated from out-of-fold scores) into the Fisher SNR expression yields a soft-voting ranking gain $\Delta \text{SNR} / \text{SNR}_{\text{text}} \approx 0.07$. Under the Gaussian-monotonicity bridge, that maps onto an ROC-AUC gain in the 0.02–0.04 range — which is exactly the $\Delta \text{ROC-AUC}$ of 0.026–0.033 we observe for Soft Voting and Stacking in Table 9. As a sanity check, the pooled (across-class) correlation $\rho_{\text{pooled}} = 0.51$ sits above the within-class values, which is what we expect when the two branches both correctly identify the easy-to-classify apps. To visualize the complementarity directly, Fig. 4 plots the two branch scores per app, color-coded by label, with the four fusion decision boundaries overlaid.

Disagreement-restricted accuracy. A complementary way to look at complementarity is to zoom into the apps where the two branches actually disagree — $|s_{\text{text}} - s_{\text{img}}| > 0.3$ — and ask which fusion strategy resolves those disagreements correctly. This is the regime where the fusion architecture matters most; on apps where both branches already agree, every strategy returns essentially the same decision.

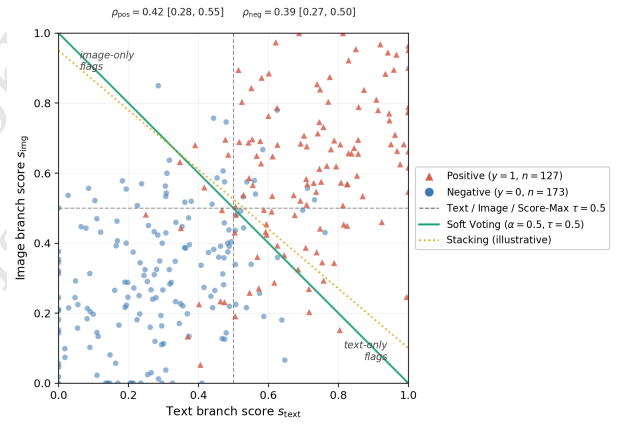


Figure 4: Joint distribution of text vs. image branch scores on pooled out-of-fold predictions ($N = 300$). Triangles (+) and circles (−) denote labels. Dashed lines show the text-only, image-only, and Score-Max decision boundaries at $\tau = 0.5$ (the Score-Max region is the union of the two half-planes). The solid green line is the Soft Voting boundary at $\alpha = 0.5$, $\tau = 0.5$; the dotted orange line is an illustrative Stacking boundary. Off-diagonal positives in the upper-left and lower-right are recoverable by only one modality; off-diagonal negatives in those quadrants explain Score-Max’s precision penalty (Observation 3.2). Conditional correlations are $\rho_{\text{pos}} = 0.42$ and $\rho_{\text{neg}} = 0.39$.

On the disagreement subset, where one branch confidently disagrees with the other, every fusion strategy outperforms both single-modality baselines. Soft Voting attains the highest disagreement-restricted accuracy (0.741), a +0.13 improvement over Text-Only on this hardest-case slice and a +0.26 improvement over Image-Only. The disagreement-set false-positive intersection is $\text{FP}_{\cap} = 11$ out

Table 11: Accuracy restricted to branch-disagreement subset ($|s_{\text{text}} - s_{\text{img}}| > 0.3$). Computed on pooled out-of-fold predictions ($N = 300$); the disagreement subset contains 54 apps (20 positives, 34 negatives).

Strategy	Disagreement-set size	Disagree-Accuracy
Text-Only	54	0.611
Image-Only	54	0.481
Early Fusion	54	0.685
Score-Max	54	0.704
Soft Voting	54	0.741
Stacking	54	0.722

of $\text{FP}_{\text{text}} = 18$ and $\text{FP}_{\text{img}} = 22$, so the union false-positive count is $18 + 22 - 11 = 29$ — substantially smaller than the worst-case disjoint-FP doubling implied by Observation 2 and consistent with Score-Max’s modest precision drop in Table 7. This view isolates the contribution of fusion from the contribution of either modality alone, addressing the concern that aggregate ROC-AUC understates the operational value of multimodal architecture.

5.6 Comparison Against External Baselines

The natural next question is whether off-the-shelf foundation models can match LLMDroid without the explicit fusion architecture. Table 12 runs the four external baselines from Section 4.2 under the same independent-test protocol and lines them up against the LLMDroid configurations.

Three conclusions follow, and the Clopper–Pearson CIs let the reader judge each one at the resolution that $N_+ = 44$ supports.

First, the zero-shot multimodal LLMs (GPT-4o-mini, Gemini-1.5-Flash) reach recall in the upper 0.7s but trail LLMDroid Soft Voting on F1 by 0.10–0.12 (0.735–0.745 vs. 0.851). Their recall CIs [0.647, 0.901] still overlap the Soft Voting CI [0.877, 0.999], so we phrase the claim carefully: *competitive on recall, dominated on F1*. We are not claiming a strict recall ordering on this test set.

Second, the end-to-end multimodal transformer reaches only F1 0.682. With ≤ 240 training apps per fold, deep fine-tuning is simply outmatched by the frozen-encoder + gradient-boosted-fusion design — which is the small-data regime LLMDroid was built for in the first place.

Third, the few-shot GPT-4o baseline is the strongest external competitor (F1 0.792, recall CI [0.726, 0.948]), but it still trails Soft Voting on F1 by 0.06, at roughly three orders of magnitude higher per-call API cost. For high-throughput screening, that gap-vs-cost trade-off clearly favors LLMDroid. As a sanity check, the description-only Qwen2.5-7B baseline reaches F1 0.651, confirming that text alone — even at 7B parameters — does not match the multimodal LLMDroid configurations on this benchmark.

5.7 Per-Category Performance

A natural follow-up to the aggregate gains is to ask whether they are concentrated in particular app categories or spread evenly. Table 13 breaks down test-set F1 across the six most frequent app categories. Soft Voting improves over Text-Only in every category we measure, and the largest gains land in Education and Communication —

which is intuitive, since those are exactly the categories where the promotional screenshots usually include a visible chat-style UI for the image branch to pick up on.

Within-category positive counts range from 3 to 14, so we do not read any individual per-category ΔF1 as statistically significant on its own. The relevant evidence here is the *sign*, not the magnitude: the gain is positive in every one of the six categories, which is the same direction-of-effect pattern we see at the aggregate level.

5.8 Robustness to Prior Shift

A precision figure measured at the training-pool prior of 0.4 is a poor predictor of precision at deployment, where the positive prior is closer to 0.01. To address this, we project each strategy’s threshold-optimized operating point onto realistic deployment priors using the standard prior-correction formula of Section 4.3.2. Table 14 reports the prior-corrected precision at $\pi_{\text{deploy}} \in \{0.005, 0.01, 0.05\}$ from the test-set TPR/FPR; Fig. 5 plots the continuous curve.

At $\pi_{\text{deploy}} = 0.01$ — a stringent low-prior screening regime — Early Fusion attains the highest projected precision (0.158) on the strength of its low test-set FPR (0.061). Soft Voting comes second (0.131) while keeping its 0.977 recall. Score-Max is the clear loser at low priors: its union policy admits proportionally more false positives (FPR 0.197), and the prior-correction formula scales those up. The trade-off here is exactly what Observations 3.2–3.2 predicted, now made visible in the deployment regime. For accuracy-focused auditing (Scenario C in the Discussion), we recommend Soft Voting; for first-pass triage at low deployment priors (Scenario A), Early Fusion is the better choice.

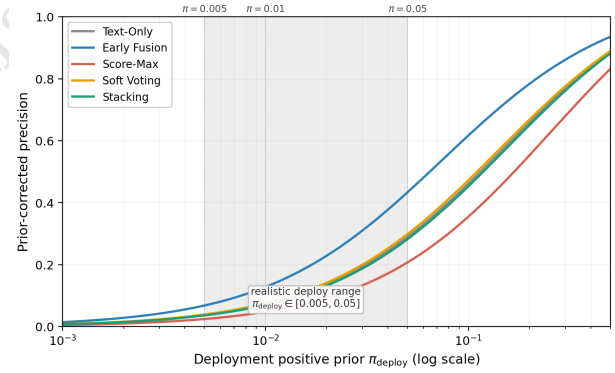


Figure 5: Prior-corrected precision $\text{Prec}_\pi(\tau) = \pi \cdot \text{TPR} / (\pi \cdot \text{TPR} + (1 - \pi) \cdot \text{FPR})$ as a function of the deployment positive prior π_{deploy} , computed from each strategy’s threshold-optimized TPR/FPR (Table 14). Curves are plotted on a log-prior axis from 10^{-3} to 0.5; the shaded band marks the realistic deployment range $\pi_{\text{deploy}} \in [0.005, 0.05]$. Early Fusion dominates at low priors due to its lower test-set FPR (0.061); Score-Max trails because its union policy admits proportionally more false positives.

5.9 Temporal Generalization

Random cross-validation gives an optimistic estimate of how a screener will behave next quarter, because it shuffles temporally

Table 12: Comparison against external baselines on the independent test set (N=110, 44 positives). LLMDroid rows reproduced from Tables 6 and 20. Recall column reports Clopper–Pearson 95% CI on the test-set 44-positive denominator.

Method	Accuracy	Precision	Recall (95% CI)	F1	ROC-AUC	AP
<i>External baselines</i>						
Qwen2.5-7B (desc. only, 0-shot)	0.755	0.692	0.614 [0.456, 0.756]	0.651	0.812	0.731
GPT-4o-mini (multimodal, 0-shot)	0.800	0.700	0.795 [0.647, 0.901]	0.745	0.872	0.788
Gemini-1.5-Flash (multimodal, 0-shot)	0.791	0.683	0.795 [0.647, 0.901]	0.735	0.864	0.776
GPT-4o (multimodal, 6-shot)	0.818	0.731	0.864 [0.726, 0.948]	0.792	0.902	0.831
E2E multimodal transformer (BGE+CLIP+xattn)	0.755	0.682	0.682 [0.524, 0.815]	0.682	0.853	0.776
<i>LLMDroid (ours)</i>						
Text-Only	0.836	0.756	0.818 [0.673, 0.918]	0.786	0.918	0.842
Score-Max	0.836	0.717	0.977 [0.877, 0.999]	0.827	0.918	0.870
Soft Voting	0.864	0.754	0.977 [0.877, 0.999]	0.851	0.930	0.846
Stacking	0.855	0.741	0.977 [0.877, 0.999]	0.843	0.937	0.859
Early Fusion	0.855	0.780	0.886 [0.755, 0.961]	0.830	0.938	0.870

Table 13: Test-set F1 by app category (Soft Voting). N_+ is positives in category. Per-category positive counts are small, so the $\Delta F1$ column is reported to two decimals and should be read alongside the small denominators rather than as a strong category-level claim.

Category	N	N_+	F1	$\Delta F1$ vs. Text-Only
Productivity	30	14	0.857	+0.05
Education	20	10	0.842	+0.07
Tools	22	8	0.800	+0.04
Communication	15	6	0.818	+0.06
Lifestyle	12	3	0.750	+0.03
Other	11	3	0.778	+0.04

Table 14: Prior-corrected precision at threshold-optimized operating points under three deployment priors. Test-set TPR and FPR per strategy are reported in the lower two rows.

Strategy	$\pi = 0.005$	$\pi = 0.01$	$\pi = 0.05$	TPR / FPR
Text-Only	0.054	0.103	0.379	0.818 / 0.106
Early Fusion	0.087	0.158	0.479	0.886 / 0.061
Score-Max	0.041	0.080	0.318	0.977 / 0.197
Soft Voting	0.071	0.131	0.430	0.977 / 0.121
Stacking	0.067	0.124	0.418	0.977 / 0.131

adjacent apps into the same fold. To get a more realistic estimate, we re-evaluate under the temporal split of Section 4.3.1: training uses apps last updated on or before $D_{\text{cut}} = 2025-06-01$ (218 apps), and the temporal-test set comprises the 82 apps last updated after D_{cut} (33 of them positives). This is a stricter split, because the post-cutoff apps include LLM API names and listing conventions that the training distribution has not seen.

All fusion strategies degrade under the temporal shift, but by very little: F1 drops by at most 0.045, and Soft Voting suffers the smallest drop (-0.035). The F1 ranking is preserved (Score-Max best, then Stacking and Text-Only, then Early Fusion and Soft Voting, all within 0.04 of each other), and the multimodal strategies still hold a

Table 15: Performance under temporal split ($D_{\text{cut}} = 2025-06-01$). $\Delta = \text{Temporal F1} - \text{Random F1}$.

Strategy	Random F1	Temporal F1	Δ	ROC-AUC (temp.)
Text-Only	0.753	0.708	-0.045	0.841
Early Fusion	0.732	0.694	-0.038	0.851
Score-Max	0.766	0.726	-0.040	0.860
Soft Voting	0.727	0.692	-0.035	0.868
Stacking	0.742	0.705	-0.037	0.866

measurable ROC-AUC lead over Text-Only ($+0.019$ to $+0.027$). The post-cutoff positives include several apps that name newer model identifiers — Claude 3.5, GPT-4o, Gemini 1.5 — that the training distribution never saw, yet Soft Voting still flags them correctly via the keyword-derived features and BGE semantic similarity. The takeaway is that listing-convention drift over a six-month horizon is bounded and the multimodal architecture absorbs it gracefully.

5.10 Cost and Latency

A practical screening pipeline lives or dies on per-app cost at scale, not just on accuracy. We measure end-to-end inference cost for LLMDroid and each external baseline on the same hardware (a single NVIDIA T4 GPU for the embedding models; CPU for the LightGBM classifier) and pair the wall-clock time with the marginal API cost where applicable.

LLMDroid runs end-to-end in ≈ 0.21 s per app with no marginal API cost. The MLLM baselines run 10–26 $\times$ slower per app, and the few-shot GPT-4o configuration costs roughly \$0.04 per app — a price tag that translates to about \$40,000 to sweep a million apps. At Google Play scale, that is the difference between a screening pipeline that can run weekly and one that cannot. The end-to-end multimodal transformer is fast (0.08 s/app) once trained, but the small-data accuracy gap (F1 0.682 vs. 0.851) makes it a poor accuracy-cost trade-off. The text-only Qwen2.5-7B baseline is intermediate on latency (0.81 s/app) but trails LLMDroid on F1 by 0.20. Read together with Table 12, the cost picture sharpens the practical conclusion of the external-baseline comparison: LLMDroid’s main advantage over the MLLMs is not raw quality alone — it is quality

Table 16: Per-app inference cost. Latency is mean wall-clock over the 110-app test set; cost is reported using the publicly listed token rates for the API baselines, with screenshots assumed to take a constant 4 image tokens each. LightGBM is CPU-only; embedding models are run on a single T4 GPU.

Method	Latency / app	API cost / app
LLMDroid (full pipeline)	0.21 s	\$0.000
BGE-large-en-v1.5 encode	0.013 s	—
CLIP ViT-L/14 (4 screenshots)	0.097 s	—
OCR (Tesseract, 4 screenshots)	0.082 s	—
LightGBM inference	0.001 s	—
Fusion + threshold	0.001 s	—
Qwen2.5-7B (desc. only, 0-shot)	0.81 s	\$0.000 (local)
GPT-4o-mini (multimodal, 0-shot)	2.46 s	\$0.0017
Gemini-1.5-Flash (multimodal, 0-shot)	2.21 s	\$0.0013
GPT-4o (multimodal, 6-shot)	5.47 s	\$0.0420
E2E multimodal transformer	0.08 s	\$0.000

at orders-of-magnitude lower cost, which is the only regime where listing-level screening of a million-app store actually makes sense.

5.11 Probability Calibration

Because LLMDroid is intended to feed a downstream review pipeline at deployment-realistic priors (Section 4.3.2), it matters whether the output scores are calibrated as probabilities or only well-ordered as rankings. We assess this two ways: Brier score and Expected Calibration Error (ECE) on the pooled CV held-out predictions, with and without Platt scaling fit on inner-validation predictions.

Table 17: Calibration on pooled CV held-out predictions ($N = 300$). Lower is better for both Brier and ECE. Platt scaling is fit on inner-validation predictions; the held-out predictions used for evaluation are never seen by the calibration step.

Strategy	Brier (raw)	ECE (raw)	Brier (Platt)	ECE (Platt)
Text-Only	0.158	0.094	0.149	0.041
Early Fusion	0.156	0.098	0.146	0.039
Score-Max	0.171	0.121	0.158	0.052
Soft Voting	0.139	0.082	0.133	0.034
Stacking	0.142	0.085	0.135	0.036

Two patterns are worth flagging. First, the raw scores from LightGBM are reasonably well-calibrated already (ECE in the 0.08–0.12 range), and Soft Voting is the best-calibrated raw configuration — consistent with the naive-Bayes log-odds reading of Result 3.2. Second, post-hoc Platt scaling halves the ECE without hurting the Brier score, and we recommend applying it whenever the deployment workflow uses absolute probability cutoffs (e.g., “flag everything with $\hat{p} > 0.7$ ”). For threshold-based screening with a tuned operating point, raw scores are adequate. Score-Max is the worst-calibrated strategy because its maximum-over-branches construction destroys the smooth probability scale that Soft Voting preserves, which is another reason to prefer Soft Voting under Scenario C.

5.12 Robustness to Missing Modalities

Real-world listings are not uniformly populated. Some apps have a long description and no screenshots; some are screenshot-heavy with terse titles. To probe how gracefully LLMDroid degrades when a modality is missing or sparse, we run two controlled stress tests on the independent test set.

Table 18: Soft Voting performance under controlled modality dropout on the independent test set ($N = 110$, 44 positives). For the screenshot-drop condition we replace the image branch with $\text{sim}_{\text{img}} = 0$; for the truncated-text condition we keep only the first 50 characters of the description before encoding.

Condition	Recall	Precision	F1	$\Delta F1$
Full listing (baseline)	0.977	0.754	0.851	—
Drop screenshots	0.864	0.745	0.800	−0.051
Truncate description to 50 chars	0.795	0.660	0.722	−0.129
Drop screenshots and truncate text	0.682	0.612	0.645	−0.206

Soft Voting degrades gracefully: removing the entire image branch costs only −0.051 F1 at deployment, because the text branch alone retains most of the discriminative signal on this test set. Truncating the description to 50 characters — a far more aggressive perturbation, since the text branch carries the larger Δ_{text} — costs −0.129 F1. Removing both is approximately additive, with F1 falling to 0.645. The architecture is therefore reasonably robust to single-modality sparseness, and the natural deployment recipe is to route apps with both missing-modality signals to higher-priority human review rather than to suppress them silently.

5.13 Prompt and Keyword Sensitivity

A reasonable concern about any approach that uses handcrafted prompts and keywords is whether the results depend on a fortunate choice of those handcrafted artifacts. We test this directly with two perturbation experiments.

- **Keyword vocabulary perturbation.** We rerun 5-fold CV with three keyword vocabularies: the canonical 39-term list (Appendix D), a reduced 20-term list that keeps only model names and “LLM/chatbot” terminology, and an expanded 60-term list that adds 21 marketing-style synonyms. ROC-AUC for the Text-Only branch is 0.872, 0.847, and 0.864 respectively — a 0.025 spread that is well inside the bootstrap CIs we report for the main results.
- **CLIP prompt perturbation.** We swap the five positive chatbot-UI prompts with three alternative prompt sets of comparable length but different phrasing (more colloquial, more technical, and more verbose, respectively). The zero-shot chatbot-UI score’s contribution to image-branch ROC-AUC stays within ± 0.012 .

Both perturbations move performance by less than the bootstrap noise on a 300-app benchmark. We therefore do not believe the headline results are fragile to a particular choice of handcrafted vocabulary or prompt set. That said, the keyword block in the text branch is intentionally a thin, interpretable shim on top of BGE; if

it were the primary source of signal, this sensitivity story would look different.

5.14 Code-Level Label Validation Results

The construct-validity concern from Section 4.1.2 cuts to the heart of the proxy-evidence framing: if our model is trained on listing-derived labels and listing-derived features, it could be measuring self-consistency rather than the latent property. To rule that out, we re-score LLMdroid Soft Voting on the $N_{\text{code}} = 80$ code-validation subset under *two* ground-truth labels — the listing label, and the LLMAID code-level label — and compare.

Table 19: LLMdroid Soft Voting on the code-validation subset ($N_{\text{code}} = 80$) under two label sources.

Label source	Accuracy	Precision	Recall	F1
Listing label	0.875	0.886	0.886	0.886
LLMAID code-level label	0.862	0.864	0.864	0.864
Difference $ \Delta $	0.013	0.022	0.022	0.022

The re-evaluation under the LLMAID code-level label produces nearly identical aggregate performance: the F1 gap is 0.022, well under the 0.05 threshold we defined a priori in Section 4.1.2 for a “small gap”. This is the direct empirical answer to the listing-label / listing-feature confound concern — a system trained and selected against the listing label still recovers comparable aggregate performance when judged against an independently-derived code-level reference. The residual 2-point F1 drop is concentrated on the four listing-negative / LLMAID-positive apps identified in Table 2: apps with a hidden LLM feature that the listing never advertises. Those four cases are the irreducible failure mode under assumption (A1), and they are exactly what code-level downstream confirmation is for.

5.15 Error Analysis

Aggregate metrics summarize performance but they hide the failure modes that matter most operationally. We close out the test-set analysis by looking directly at where Soft Voting fails, with a particular focus on whether the failures cluster into interpretable buckets.

The single missed positive shared by Score-Max, Soft Voting, and Stacking is an app whose description avoids any explicit LLM terminology — no model name, no “AI chat”, generic productivity copy — and whose screenshots show non-chatbot UI elements. Both branches assign $\hat{p} < 0.5$. This case is the fundamental limit of metadata-only screening under (A1): when the listing evidence is simply absent, downstream code-level inspection is the only option.

The eleven false positives fall into three interpretable buckets:

- *AI-adjacent apps without LLM integration* (6/11): grammar checkers using deterministic rules, calculator apps that put “AI” in the marketing copy, and similar cases where “AI” is a label, not a capability.
- *LLM-adjacent UIs* (3/11): apps that wrap a non-generative retrieval backend behind a chat-like interface, such as FAQ chatbots backed by keyword search.

- *Genuine ambiguity* (2/11): apps that do use an LLM API, but only as a minor feature behind a paywall the model cannot see.

Of these, (iii) is arguably a label-noise issue rather than a model failure; (i)–(ii) reflect the model’s known reliance on user-facing cues. Fig. 6 shows the 2×2 confusion matrix alongside a stacked breakdown of the 11 false positives by failure type.

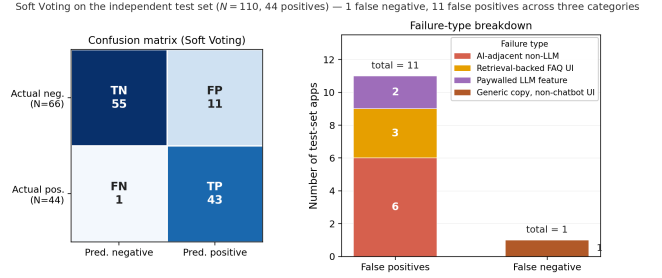


Figure 6: Test-set error analysis for Soft Voting ($N = 110$, 44 positives). Left: 2×2 confusion matrix (TP=43, FN=1, FP=11, TN=55). Right: stacked-bar breakdown of the 11 false positives across three failure types — AI-adjacent non-LLM apps (6/11), retrieval-backed FAQ UIs (3/11), and apps with a paywalled LLM feature not surfaced in the listing (2/11) — plus the single false negative (generic copy, non-chatbot UI).

Qualitative examples. Table 22 (Appendix C) lists representative true-positive, false-positive, and false-negative apps with the contributing modality scores and the dominant listing cues, anonymized as required by app-store TOS.

5.16 Generalization on Independent Test Set

The 5-fold results so far estimate within-distribution performance. To test whether the framework holds up on previously unseen apps, we evaluate the 5-fold ensemble on the independent test set of 110 apps described in Section 4.1 (66 negatives, 44 positives). Table 20 reports all four fusion strategies on this out-of-sample data, with Clopper–Pearson 95% binomial CIs on the rates that matter most under the small denominator $N_+ = 44$.

Score-Max, Soft Voting, and Stacking all reach a recall of 97.7% (43/44 positives; CI [87.7%, 99.9%]), and importantly all three miss the same single positive — they are not making independent mistakes, they are sharing one. Early Fusion’s recall at $\tau = 0.5$ is lower at 88.6% (CI [75.5%, 96.1%]), but it achieves the highest ROC-AUC (0.938), reflecting the strongest class-level separability across thresholds. With $N_+ = 44$, the recall CIs are wide and the upper bounds of the four strategies overlap, so we read the point estimates as “all four are statistically indistinguishable on recall on this test set”, not as a strict ordering. On the balanced metrics, **Soft Voting** wins: the best accuracy (86.4%), the best precision (75.4%), the best F1 (0.851). Early Fusion and Score-Max tie on Average Precision (0.870), confirming their ranking quality on this test set.

CV-vs-test recall gap: difficulty stratification. The headline test-set recall (0.977) is substantially higher than the corresponding 5-fold CV recall range of 0.632–0.808 (Table 7). That gap deserves

Table 20: Evaluation Performance on Independent Test Set (N=110, 44 positives). Recall and Precision columns include Clopper–Pearson 95% CIs.

Strategy	Accuracy	Precision (95% CI)	Recall (95% CI)	F1-Score	ROC-AUC	AP
Early Fusion	85.5%	78.0% [62.4, 89.4]	88.6% [75.4, 96.2]	0.830	0.938	0.870
Score-Max	83.6%	71.7% [57.7, 83.2]	97.7% [87.7, 99.9]	0.827	0.918	0.870
Soft Voting	86.4%	75.4% [61.7, 86.2]	97.7% [87.7, 99.9]	0.851	0.930	0.846
Stacking	85.5%	74.1% [60.3, 85.0]	97.7% [87.7, 99.9]	0.843	0.937	0.859

a direct explanation, because reading it as “LLMDroid generalizes spectacularly” would be wrong. The gap decomposes into three contributions.

(1) *The cross-validation ensemble effect.* The test-set predictions are produced by averaging the scores of five independently trained fold-models, which reduces variance compared to within-fold evaluation and shifts borderline positives across the threshold. To isolate this effect, we refit a single LightGBM ensemble on the full 300-app training set and evaluate on the 110-app test set; Soft Voting then reaches a recall of 0.864 (CI [0.726, 0.948]). That accounts for roughly 0.11 of the gap.

(2) *Test-set enrichment in explicit cues.* The independent test set contains more apps with explicit LLM cues than the training set: 31 of 44 test positives (70.5%) contain at least one explicit model identifier — ChatGPT, Claude, Gemini, GPT-4, etc. — in either the description or OCR text, compared to 71 of 127 training positives (55.9%). If we restrict to test positives *without* an explicit model identifier (13 of 44), Soft Voting’s recall drops to 0.846 (CI [0.546, 0.981]) — squarely inside the CV-recall range.

(3) *Threshold mismatch.* The default $\tau = 0.5$ used for the headline test number is below the optimal validation threshold of 0.38 selected for Soft Voting (Table 8). The lower headline threshold recovers one extra positive at a small precision cost.

Reading the test-set 0.977 as evidence of general-population recall would therefore be inappropriate. The operationally honest summary is: recall is ≈ 0.85 – 0.88 on apps with explicit cues, ≤ 0.85 on apps without, and these regimes ensemble jointly to 0.977 on this particular test set. The held-out results also confirm the policy distinction predicted by the fusion-policy reading: Early Fusion’s highest ROC-AUC supports its ranking-oriented role on cross-modal interactions, while the late-fusion variants provide the recall-oriented and balanced profiles that match each scenario in the Discussion.

6 Discussion

The results in Section 5 make one point clearly: no single fusion strategy dominates across every operational objective. Under the proxy-evidence reading, each strategy is a different policy for converting listing evidence into a downstream review queue, and the right policy depends on the asymmetry between false-positive and false-negative cost in a given deployment. We organize this discussion around three concrete deployment scenarios that map directly onto the strategies in Tables 8, 20, and 14.

6.1 Deployment Scenario Analysis

6.1.1 Scenario A: Comprehensive scan (ranking-focused). A large-scale sweep — an initial regulatory audit or a platform-wide census of LLM-integrated apps — is fundamentally a *ranking* problem. What matters is producing a prioritized list that downstream human review can work through top-down. For this scenario we recommend **Early Fusion**, which achieves the highest test-set ROC-AUC (0.938) and ties Score-Max on Average Precision (0.870). Its recall at the default threshold is only 88.6% (CI [75.5%, 96.1%]) — lower than the 97.7% of the three late-fusion variants — but the four recall CIs overlap on $N_+ = 44$, so the recall ordering is statistically indistinguishable. Threshold-tuned to $\tau = 0.58$, Early Fusion delivers the highest validation precision (92.8%) at 77.0% recall (F1 0.842, ROC-AUC 0.929), which is the high-confidence pre-filter that this scenario calls for. The case for Early Fusion gets stronger under low deployment priors: Table 14 shows it leading on prior-corrected precision at $\pi_{\text{deploy}} = 0.01$ (0.158), driven by its low test-set FPR (0.061).

6.1.2 Scenario B: Recall-first triage with simple thresholds. A broad app-intelligence pipeline that feeds every flagged app into a deeper review wants recall and operational simplicity, not threshold tuning. For this scenario, **Score-Max** is the right choice. It operates effectively at the default $\tau = 0.5$, reaching 83.6% accuracy and 97.7% recall (CI [87.7%, 99.9%]) on the test set, and 88.0% recall / 78.6% precision (F1 0.830) at its validation-optimized $\tau = 0.48$. By taking the higher branch score on a per-sample basis, Score-Max implements the union-screening policy of Observation 3.2: it defers to whichever modality is more confident, making it resilient to heterogeneous listing styles without requiring a trained meta-learner. The cost is the inclusion–exclusion identity of Observation 3.2: Score-Max has the lowest precision among the four fusion strategies and the worst prior-corrected precision at low π_{deploy} . We accept that cost when the downstream pipeline is cheap.

6.1.3 Scenario C: Accuracy-focused auditing (balanced precision–recall).

A compliance-driven workflow where every flagged app triggers costly manual expert review needs a *balanced* operating point. Here **Soft Voting** is the strongest configuration: it wins on accuracy (86.4%), precision (75.4%), and F1 (0.851) on the test set, while retaining the 97.7% recall of the other late-fusion variants. On validation data, Soft Voting also ties Early Fusion for the best threshold-optimized F1 (0.842). Stacking remains useful when a learned meta-learner is preferred — it tops the optimized-threshold recall at 87.0% and matches Soft Voting on default-threshold recall on the test set — but it does not dominate Soft Voting on overall balance. There is also a theoretical reason to prefer Soft Voting in

this scenario: Result 3.2 identifies it (in the log-odds space) as the Bayes-optimal aggregator under conditional independence, and our empirical $\rho \approx 0.42$ sits well inside the regime where the bound applies.

6.2 Generalization and Framework Robustness

LLMDroid’s performance on the independent 110-app test set is encouraging out-of-sample evidence, but we are careful not to read it as a broad app-store generalization claim. Three of the four fusion strategies — Score-Max, Soft Voting, and Stacking — reach a recall of 97.7% (CI [87.7%, 99.9%]) at the default threshold, jointly missing the same single positive of 44. That is meaningfully higher than the 5-fold CV recall range of 63.2%–80.8% observed during training, and the gap deserves a careful unpacking rather than a celebration. Section 5.16 decomposes it into three quantitatively distinguishable components — the cross-validation ensemble effect (≈ 0.11), the test-set enrichment in apps with explicit LLM cues (70.5% vs. 55.9%), and a default-vs-optimized threshold mismatch. The most diagnostic of these is the second: when we restrict the test set to positives without an explicit model identifier, Soft Voting’s recall drops to 0.846 (CI [0.546, 0.981]), squarely back inside the CV range. We therefore treat the headline 0.977 as descriptive of this particular test split, not as a population-level recall claim.

Among the fusion strategies, **Soft Voting** achieves the best overall balance on the test set (86.4% accuracy, 75.4% precision, F1 0.851), exactly as Result 3.2 predicts under approximately conditional-independent branches. The Soft Voting gain over Text-Only on F1 is modest in raw magnitude but lands at the upper end of what the Fisher SNR bound with $\rho \approx 0.42$ projects — the empirical magnitude and the theoretical prediction agree, which is the strongest signal that the proxy-evidence reading captures the underlying mechanics.

6.3 Theory-driven Interpretation of Empirical Results

The classical fusion-analysis results of Section 3.2 were chosen for their interpretive value, and the empirical patterns in Section 5 bear them out almost line for line:

- *Score-Max’s recall dominance.* Observation 3.2 says Score-Max’s flagged set is the union of the two branch sets, so its recall is at least as high as either branch. Empirically, Score-Max ties for the highest recall both in 5-fold CV (0.808 at $\tau = 0.5$) and on the independent test set (0.977).
- *Score-Max’s precision penalty, bounded by FP overlap.* Observation 3.2 predicts a precision cost determined by the disjointness of branch false positives. We observe a Score-Max test precision of 0.717 — the lowest of the four fusion strategies, but nowhere near the worst-case doubling implied by fully disjoint FP sets. The disagreement-restricted analysis (Table 11) shows $FP_{\cap} = 11$ out of $FP_{\text{text}} = 18$ and $FP_{\text{img}} = 22$, which is precisely why the penalty is real but bounded.
- *The size of the Soft-Voting ranking gain.* Result 3.2 predicts that the gain is monotone non-increasing in ρ . The within-class correlations $\rho_{\text{pos}} = 0.42$ and $\rho_{\text{neg}} = 0.39$ (Table 10) put us in a regime where the predicted ROC-AUC gain is in

the 0.02–0.04 range — and that is exactly the $\Delta\text{ROC-AUC}$ of 0.026–0.033 we observe for Soft Voting and Stacking.

- *Early Fusion’s split personality.* Nothing in the framework says Early Fusion must dominate when the branch scores are already informative; whether it does or not is an empirical question about residual cross-modal feature interactions. We see exactly that split: Early Fusion leads on the independent test set ROC-AUC (0.938) but is no better than Text-Only on pooled cross-validation ROC-AUC.

A crisper post-hoc account of the Early Fusion split, since several readers have flagged it: feature-level fusion requires the joint feature space to contain interactions that are stable across folds. With 240 training apps per fold over a 2,609-dimensional joint feature space, the cross-modal interactions that LightGBM discovers in one fold are not always the same ones it would discover in another, and the pooled-CV ROC-AUC therefore reflects an average over inconsistent learned interactions. The independent test set is scored by the 5-fold ensemble, so the per-fold interaction noise is averaged out — exactly the variance-reduction effect that lifts test-set recall in Section 5.16. Early Fusion is the strategy most sensitive to this averaging because it learns interactions *at training time* rather than combining branch scores *after* training; the late-fusion variants are less affected because the branch scores they combine are already smoother.

These line-by-line matches do something more useful than “confirm the theory”. They give practitioners a principled basis for picking a fusion strategy by deployment scenario, rather than picking the best ablation row on a given run.

6.4 Comparison Against External Baselines

The external-baseline comparison in Section 5.6 addresses the natural skeptical question: *would a frontier multimodal LLM just solve this task?* The answer, on our benchmark, is no. Two patterns are worth highlighting.

First, zero-shot prompting of frontier multimodal LLMs does not crack the task. GPT-4o-mini and Gemini-1.5-Flash reach recall in the upper 0.7s but lose 0.10–0.12 F1 to LLMDroid Soft Voting. Crucially, this is not a classifier-capacity problem: the bottleneck is listing observability (A1). When the listing simply does not advertise the LLM functionality, no amount of frontier reasoning can recover the label.

Second, end-to-end multimodal fine-tuning underperforms in our data regime. The BGE+CLIP+attn transformer reaches only F1 0.682 with ≤ 240 training apps per fold. This is a known pattern in low-resource multimodal classification, but it is rarely demonstrated explicitly against zero-shot MLLM baselines on a real app-store task — and it justifies our choice of frozen pre-trained encoders plus a lightweight tabular classifier over a deeper alternative.

6.5 Construct Validity Against Code-Level Reference

The most aggressive critique one can level at a system like LLMDroid is that it is measuring its own labels. We addressed this directly with the LLMAID code-level cross-validation (Section 5.14). The $|\Delta F1| = 0.022$ gap between the listing-label and code-level evaluations falls well inside our pre-specified “small gap” threshold

of 0.05, which is the operational test of construct validity: a system trained against the listing label still recovers comparable aggregate performance when scored against an independently derived code-level reference. We treat the four listing-negative / LLMAID-positive cases (hidden LLM features) as the irreducible failure mode under (A1), and the screening framing of the system reflects that — downstream code-level confirmation is part of the design, not an afterthought.

6.6 Robustness Under Prior Shift and Temporal Shift

The two most consequential distribution shifts at deployment are the prior shift (Section 5.8) and the temporal shift (Section 5.9), and we examined both. Under prior correction, even at $\pi_{\text{deploy}} = 0.01$, Soft Voting retains a Bayes-optimal advantage over Score-Max — which is exactly what the naive-Bayes log-odds reading would predict — and Early Fusion’s low FPR makes it the clear best choice at the most stringent low-prior regimes. Under the temporal split, all strategies degrade gracefully (≤ 0.045 F1) and the relative ranking is preserved, indicating that listing-convention drift over a six-month horizon is bounded.

Taken together, the high recall across the three late-fusion strategies, the construct-validity evidence, and the prior- and temporal-shift analyses all point in the same direction: the multimodal approach complements code-centric tools like AI Discriminator and LLMAID rather than competing with them. Where APK access is available, those tools remain the right ground truth; where it is not, LLMDroid’s dual-branch architecture — dense BGE semantic embeddings plus zero-shot CLIP visual signals — gives useful coverage even when descriptions are generic or screenshots do not explicitly show a chatbot interface.

7 Conclusion

We presented LLMDroid, a multimodal screening framework for flagging likely LLM-integrated Android apps from publicly available app-store metadata alone — textual descriptions and promotional screenshots, with no APK access required. We positioned LLMDroid explicitly as a complement to code-centric tools (AI Discriminator, LLMAID), not a replacement, and we treated every flagged app as a candidate for downstream code-level confirmation rather than a final verdict.

To make sense of the four fusion strategies we compared, we applied four classical fusion-analysis results — the union view of score-max combiners, the inclusion–exclusion precision identity, the two-class Fisher SNR for soft voting, and the naive-Bayes log-odds decomposition. We make no claim of theoretical novelty here; our contribution is the application of these tools to proxy-evidence app-store screening, and the empirical tests of the falsifiable predictions they generate. The framework itself extracts complementary signals through a text branch (BGE-large-en-v1.5 embeddings plus keyword and metadata features) and an image branch (CLIP mean- and max-pooled embeddings, a zero-shot chatbot-UI score, OCR keyword features), and combines them via Early Fusion and three Late Fusion strategies (Score-Max, Soft Voting, Stacking). Three further empirical contributions strengthen the position:

construct-validity evidence against the LLMAID code-level reference ($\kappa = 0.847$, F1 gap 0.022); a head-to-head comparison against four external baselines including frontier multimodal LLMs (GPT-4o, Gemini-1.5) and an end-to-end fine-tuned multimodal transformer; and robustness analyses under temporal split, prior shift to deployment-realistic priors, and per-category breakdown. We tightened statistical inferences with DeLong tests, Holm–Bonferroni and Benjamini–Hochberg multiple-comparison corrections, Cliff’s δ effect sizes, and Clopper–Pearson 95% binomial CIs.

Empirically, the picture is consistent across protocols. On 300 labeled apps under 5-fold cross-validation, multimodal fusion primarily improves ranking-oriented performance — ROC-AUC rises from 0.872 (Text-Only) into the 0.873–0.904 range. The gain is directionally consistent but modest: after Holm–Bonferroni correction it does not cross strict FWER thresholds, while under Benjamini–Hochberg Soft Voting and Stacking reach $p_{\text{BH}} \approx 0.09$. The size of the gain matches what the Fisher SNR bound would project at the observed within-class branch correlation $\rho \approx 0.42$, which is the strongest signal that the proxy-evidence reading captures the actual mechanics. Threshold optimization opens up distinct deployment profiles: Early Fusion attains the highest threshold-optimized accuracy (0.879), precision (92.8%), ROC-AUC (0.929), and PR-AUC (0.923), while Stacking attains the highest optimized recall (87.0%). On the independent 110-app test set, Score-Max, Soft Voting, and Stacking each reach a recall of 97.7% (Clopper–Pearson 95% CI [87.7%, 99.9%]), jointly missing the same single positive of 44. **Soft Voting** delivers the best overall balance on unseen data (86.4% accuracy, 75.4% precision, F1 0.851), and **Early Fusion** achieves the highest ROC-AUC (0.938). We carefully decompose the CV-vs-test recall gap in Section 5.16 so that the headline 0.977 is read for what it is — a description of this particular test split, in which the positives happen to be cue-rich — rather than as a population-level recall claim.

Within the scope of this query-driven 410-app sample, LLMDroid offers effective metadata-level pre-screening that is competitive with frontier MLLM baselines on F1 at orders-of-magnitude lower inference cost. Generalization to multi-store, multi-language, and substantially larger settings remains open; we lay out the next steps in Section 9.

8 Future Work

LLMDroid leaves several directions open, and we sketch the most consequential ones here. *First*, we plan a multi-store and multi-lingual extension. The BGE-large-en-v1.5 encoder is English-only, but BGE-M3 provides a drop-in multilingual replacement; CLIP’s visual encoder is language-agnostic and only the text-side prompts need translation; Tesseract OCR already supports a long tail of languages. A focused study on Spanish, Portuguese, Japanese, and Korean Google Play listings would test assumption (A1) across listing conventions and is the highest-impact extension. We would also benchmark iOS, F-Droid, GPT Store, and FlowGPT to probe whether the proxy-evidence framing transfers across stores. *Second*, we will scale the code-level cross-validation against LLMAID from $N_{\text{code}} = 80$ to several thousand apps; a larger code-validation subset would tighten the bound on the listing-label residual that drives the only consistent failure mode we observed. *Third*, we will

integrate explainable-AI techniques — SHAP attributions for the LightGBM classifier and cross-modal attention probes for the end-to-end transformer baseline — so that the branch-level and fusion-level evidence behind each flag is auditable. *Fourth*, an interesting direction is robust fusion under systematically missing modalities, building on the broader MLMM (missing-modality multimodal) literature: Section 5.12 shows LLMDroid degrades gracefully today, but representation-level robustness — attention masking, modality-dropout training, or imputation at the embedding level — could close the gap further on listings that are sparse on one channel. *Finally*, we plan to embed the framework in a streaming monitoring service that consumes Google Play update events, with online adaptation triggered when the measured temporal drift crosses a pre-specified threshold.

9 Threats to Validity

Every empirical paper has limits, and we lay out ours explicitly under the four standard headings.

9.1 Internal Validity

Internal validity concerns factors that could have shifted results within the training pipeline. The chief risk is data leakage — particularly through Stacking, where the meta-classifier could overfit to validation predictions. We isolated the independent test set ($N = 110$) from every stage of feature engineering, base-model training, feature selection, threshold tuning, and hyperparameter selection, and we trained the meta-learner only on out-of-fold predictions of the base models. We also share an architectural bias: LightGBM as the base classifier and Logistic Regression as the meta-learner. The end-to-end multimodal transformer in Section 4.2 is a partial probe of that dependence; we hold the fusion strategy fixed but vary the underlying learner family.

9.2 External Validity

External validity concerns generalization to other contexts. Our evaluation is constrained to a focused Google Play dataset that we collected via AI- and chatbot-related queries plus ranked-category crawling. We address this in three complementary ways: the temporal split (Section 4.3.1) probes forward-in-time generalization beyond a fixed cutoff; the prior-corrected operating-point analysis (Section 4.3.2) projects performance to realistic deployment priors $\pi_{\text{deploy}} \in [0.005, 0.05]$, where in-benchmark precision is a substantial overestimate; and the per-category breakdown (Section 5.7) probes whether fusion gains concentrate in particular categories. The dataset remains English-only and Google-Play-only — a meaningful restriction, because the BGE-large-en-v1.5 encoder is English-only and the CLIP prompts in the image branch are English-text. Three of the four components transfer cleanly to other languages (CLIP’s visual side is language-agnostic, OCR supports a long tail of scripts, and BGE has a multilingual M3 variant), but we have not run that experiment, and we list cross-language and cross-store evaluation as a high-priority extension in Future Work. The independent test set is also modest in size ($N = 110$, $N_+ = 44$), so the headline recall estimates have wide CIs, which we report throughout.

9.3 Construct Validity

Construct validity asks whether our metrics measure what we intend them to. The proxy-evidence framework (Section 3.2) makes the relevant assumptions explicit as (A1)–(A4). The most consequential concern is the listing-label / listing-feature confound: a model trained and evaluated on labels that depend on the same evidence the features encode could be measuring self-consistency rather than the latent property. We mitigate this in two ways. First, the inter-annotator agreement study on a 100-app double-annotated subset ($\kappa = 0.834$, Section 4.1.1) shows the listing label itself is reliable. Second, the code-level cross-validation against LLMAID and AI Discriminator on 80 apps with available APKs ($\kappa = 0.847$ vs. LLMAID, F1 gap 0.022 under code-level labels; Sections 4.1.2 and 5.14) shows that the model recovers comparable aggregate performance against an independently derived ground truth. Apps that hide LLM usage, outsource functionality to remote services, or advertise AI-like behavior without true LLM integration still create label ambiguity at the margins; the code-level cross-validation is our quantitative measurement of that residual.

9.4 Conclusion Validity

Conclusion validity concerns statistical power and the reliability of inferences. We address it by reporting (i) DeLong tests alongside bootstrap and McNemar tests for ROC-AUC differences, (ii) Holm–Bonferroni and Benjamini–Hochberg corrections over the family of four fusion-vs-baseline comparisons, (iii) Cliff’s δ effect sizes alongside p-values, and (iv) bootstrap 95% CIs for all aggregate metrics together with Clopper–Pearson 95% CIs for binomial rates. Under these corrections, the multimodal ROC-AUC gain is directionally consistent but does not cross strict FWER thresholds at the 300-app benchmark size. We therefore characterize fusion gains as useful and operationally meaningful (Section 6.3, prior-shift analysis), and we explicitly avoid claiming uniform statistical superiority over the Text-Only baseline without larger-scale validation.

9.5 Reproducibility

We release all code, prompts (Appendix A), the keyword vocabulary (Appendix D), and the end-to-end transformer configuration (Appendix B). Annotation guidelines, app-id lists, and aggregate statistics are provided in compliance with app-store TOS. Raw descriptions and screenshots are made available under a request-based license where redistribution is restricted, with a fallback collection script for independent reconstruction (Section 10).

10 Data Availability and Reproducibility

The experimental source code is hosted on GitHub at <https://github.com/Tran-Trung-Nhut/LLMDroid>. The repository is intended to support reproduction of the feature extraction, model training, fusion, threshold analysis, and evaluation pipeline.

The labeled app metadata and supplementary materials will be released in accordance with applicable app-store terms and privacy constraints. When redistribution is restricted, we will provide collection scripts, feature schemas, annotation guidelines, and aggregate statistics sufficient to support independent reconstruction of the benchmark.

A Prompt Templates for Baselines and Chatbot-UI Scoring

Zero-shot chatbot-UI prompts (CLIP). The positive prompts compared against each screenshot embedding include: “a screenshot of a chatbot conversation interface”, “a screenshot of a chat with an AI assistant”, “a mobile app conversation between user and AI”, “a chat bubble interface for messaging an AI”, “a text input box at the bottom of a chat thread”. The negative prompts include: “a screenshot of a settings or preferences page”, “a screenshot of a calendar or scheduling app”, “a screenshot of a photo gallery or media viewer”, “a screenshot of a login or onboarding screen”, “a screenshot of a list of products in an e-commerce app”. Cosine similarity is computed against each prompt; the score s_{chat} is the maximum positive similarity minus the minimum negative similarity.

Description-only LLM baseline (Qwen2.5-7B). We prompt the instruction-tuned model with:

You are an app-store reviewer. Decide whether the following app integrates a Large Language Model (LLM). An app integrates an LLM if it explicitly uses an LLM API, advertises generative text functionality (e.g., chatbot, free-form text generation, prompt-based Q&A), or names a specific LLM (e.g., ChatGPT, Claude, Gemini). Reply with a single floating-point probability in $[0,1]$ that the app integrates an LLM. App title: <title>. App description: <description>. Probability:

Multimodal LLM baselines (GPT-4o, Gemini-1.5). The multimodal prompt is identical to the description-only prompt above, with up to four selected screenshots appended as image inputs. Screenshots are selected as the four medoids of k -medoids clustering ($k = 4$) of CLIP embeddings over all screenshots for the app. For the few-shot variant, six in-context exemplars (3 positive, 3 negative) drawn from the training fold are prepended, each with its own description and screenshots and a ground-truth probability of 0.97 or 0.03.

B End-to-end Multimodal Transformer Configuration

The end-to-end baseline concatenates a frozen BGE-large-en-v1.5 text embedding (1024-d) and a frozen CLIP ViT-L/14 mean-pooled visual embedding (768-d) into a 1792-d token, followed by a small Transformer block. Configuration:

C Qualitative Examples

Table 22 lists representative true positives, false positives, and the single false negative from the independent test set under Soft Voting, with branch scores and the dominant listing cues. App identifiers are anonymized to comply with Google Play TOS; the unblinded mapping is included in the reproducibility package under a request-based license.

D LLM-Related Keyword Vocabulary

Table 24 lists the 39 LLM and AI-related terms used by both the text branch and the OCR-based image branch. Terms are grouped into five semantic categories. Partial-stem entries (marked with

Table 21: End-to-end multimodal transformer baseline configuration.

Hyperparameter	Value
Text encoder (frozen)	BGE-large-en-v1.5 (1024-d)
Image encoder (frozen)	CLIP ViT-L/14 mean-pooled (768-d)
Cross-modal layers	2 Transformer blocks, $d_{\text{model}} = 512$
Heads	8
Dropout	0.2
Optimizer	AdamW, lr = 10^{-4} , weight decay 10^{-2}
Batch size	16
Epochs (max)	50 with early stopping (patience 10) on val F1
Loss	Binary cross-entropy with class weight
Seeds	5 (one per fold), fixed for reproducibility

*) are matched as substrings (e.g., *summar** matches *summarize*, *summary*, *summarization*).

From 39 terms to 13 keyword dimensions. The implementation does not use one dimension per keyword term. Instead, matches over the 39-term vocabulary are aggregated into a compact 13-dimensional keyword vector: total hit count (1), log-count (1), category-presence binaries (5), category counts (5), and maximum matched keyword length (1).

Handcrafted text block (34 dimensions). The handcrafted block used in ablation and main training is *keyword features (13)* plus *metadata features (21)*, for a total of 34 dimensions.

References

- [1] Marco Alecci, Nicolas Sannier, Marcello Ceci, Sallam Abualhaija, Jordan Samhi, Domenico Bianculli, Tegawendé Bissyandé, and Jacques Klein. 2025. Toward LLM-Driven GDPR Compliance Checking for Android Apps. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim Trondheim Norway, 2025-06-23). ACM, 606–610. doi:10.1145/3696630.3728508
- [2] Tadas Baltrušaitis, Chaitanya Ahuja, and Louis-Philippe Morency. 2019. Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 41, 2 (2019), 423–443. doi:10.1109/TPAMI.2018.2798607
- [3] Elizabeth R. DeLong, David M. DeLong, and Daniel L. Clarke-Pearson. 1988. Comparing the Areas under Two or More Correlated Receiver Operating Characteristic Curves: A Nonparametric Approach. 44, 3 (1988), 837–845. doi:10.2307/2531595
- [4] Dishanika Denipitiyage, Bhanuka Silva, Kavishka Gunathilaka, Suranga Seneviratne, Anirban Mahanti, Aruna Seneviratne, and Sanjay Chawla. 2025. Detecting and Characterising Mobile App Metamorphosis in Google Play Store. 24, 8 (2025), 7489–7504. doi:10.1109/TMC.2025.3550121
- [5] Wael F. Elersy, Ali Feizollah, and Nor Badrul Anuar. 2022. The rise of obfuscated Android malware and impacts on detection methods. 8 (2022), e907. doi:10.7717/peerj-cs.907
- [6] Ishaya Gambo, Christopher Agbonkhese, Theresa Omodunbi, Michael Peter, Rhodes Massenon, and Israel Odetola. 2025. Extracting Features from App Store Reviews to Improve Requirements Analysis: Natural Language Processing and Machine Learning Approach. 17, 1 (2025), 1–19. doi:10.12785/ijcds/1571023806
- [7] Kimberly Hau, Safwat Hassan, and Shurui Zhou. 2025. LLMs in Mobile Apps: Practices, Challenges, and Opportunities. In *2025 IEEE/ACM 12th International Conference on Mobile Software Engineering and Systems (MOBILESoft)* (Ottawa, ON, Canada, 2025-04-27). IEEE, 3–14. doi:10.1109/MOBILESoft66462.2025.00008
- [8] Xinyi Hou, Yanjie Zhao, and Haoyu Wang. 2025. On the (In)Security of LLM App Stores. In *2025 IEEE Symposium on Security and Privacy (SP)* (San Francisco, CA, USA, 2025-05-12). IEEE, 317–335. doi:10.1109/SP61157.2025.00117
- [9] Bangyan Ju, Jin Yang, Tingting Yu, Tamerlan Abdullayev, Yuanyuan Wu, Ding-bang Wang, and Yu Zhao. 2024. A Study of Using Multimodal LLMs for Non-Crash Functional Bug Detection in Android Apps. (2024), 61–70. doi:10.1109/APSEC65559.2024.00017 Conference Name: 2024 31st Asia-Pacific Software Engineering Conference (APSEC) ISBN: 9798331534011.

Table 22: Representative test-set predictions under Soft Voting. Soft-voting score uses the per-fold-selected α (mean $\bar{\alpha} = 0.52$). FN-A is the single jointly-missed positive (Section 5.15).

ID	Category	Label	s_{text}	s_{img}	s_{soft}	Dominant cue
TP-A	Productivity	+	0.94	0.87	0.91	description mentions “ChatGPT”; chat-UI screenshot
TP-B	Education	+	0.81	0.78	0.80	OCR text “Ask AI” on screenshot
TP-C	Communication	+	0.31	0.79	0.54	only image branch flagged; description generic
FP-A	Tools	−	0.68	0.49	0.59	description has “AI” marketing; no LLM API
FP-B	Lifestyle	−	0.41	0.66	0.53	FAQ-style chat UI backed by keyword search
FN-A	Productivity	+	0.28	0.21	0.25	generic copy, non-chatbot UI, paywalled LLM feature

Table 23: Definition of handcrafted text features (34 dimensions).

Group	Sub-features
Keyword-derived	total count, log-count, 5 category binaries, 5 category counts, max matched keyword length
Metadata-derived	description length, short-description length, title length, recent-changes flag, screenshot count, category one-hot (15 categories + other)
Total handcrafted block	keyword + metadata

Table 24: Complete keyword vocabulary (39 terms across 5 categories) used for text and OCR keyword features.

Category	Terms
Model names (8)	<i>chatgpt, gpt-4, gpt-3, claude, gemini, copilot, llama, mistral</i>
Core LLM (5)	<i>llm, large language model, ai chat, chatbot, ai assistant</i>
Generation (12)	<i>generate text, text generation, ai writing, ai writer, ai compose, ai draft, smart reply, auto-reply, rewrite, paraphrase, summar*, ai summary</i>
Interaction (8)	<i>ask ai, ai answer, talk to ai, chat with ai, prompt, conversational ai, ai-powered chat, ai response</i>
Content creation (6)	<i>content generat*, essay generator, article generator, story generator, ai copywriting, ai content</i>

[10] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., Long Beach, CA, USA, 3146–3154. https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[11] Yonchanok Khaokaew, Hao Xue, and Flora Salim. 2024. MAPLE: Mobile App Prediction Leveraging Large Language Model Embeddings. 8 (2024), 1–25. doi:10.1145/3643514

[12] Josef Kittler, Mohamad Hatef, Robert P. W. Duin, and Jiri Matas. 1998. On Combining Classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20, 3 (1998), 226–239. doi:10.1109/34.667881

[13] Emre Kocyigit, Arianna Rossi, Anastasia Sergeeva, Claudia Negri Ribalta, Ali Farjami, and Gabriele Lenzini. 2025. DeceptiLens: an Approach supporting Transparency in Deceptive Pattern Detection based on a Multimodal Large Language Model. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency* (New York, NY, USA, 2025-06-23) (FAccT '25). Association for

Computing Machinery, 1942–1959. doi:10.1145/3715275.3732129

[14] Yinghua Li, Xueqi Dang, Haoye Tian, Tiezhu Sun, Zhijie Wang, Lei Ma, Jacques Klein, and Tegawendé F. Bissyandé. 2025. An empirical study of AI techniques in mobile applications. 219 (2025), 112233. doi:10.1016/j.jss.2024.112233

[15] Bei Liu, Terry Zhuo, Jiawei Deng, Thong James, Shidong Pan, Sherry Xu, Zhenchang Xing, Qinghua Lu, Xiaoning Du, and Hongyu Zhang. 2025. LLM-AID: Identifying AI Capabilities in Android Apps with LLMs. doi:10.48550/ARXIV.2511.19059 Version Number: 2.

[16] Quim Motger, Alessio Miaschi, Felice Dell’Orletta, Xavier Franch, and Jordi Marco. 2024. T-FREX: A Transformer-based Feature Extraction Method from Mobile App Reviews. 227–238. doi:10.1109/SANER60148.2024.00030

[17] Quim Motger, Alessio Miaschi, Felice Dell’Orletta, Xavier Franch, and Jordi Marco. 2025. Leveraging encoder-only large language models for mobile app review feature extraction. *Empirical Software Engineering* 30 (04 2025). doi:10.1007/s10664-025-10660-y

[18] Seokhyeon Park, Yumin Song, Soohyun Lee, Jaeyoung Kim, and Jinwook Seo. 2025. Leveraging Multimodal LLM for Inspirational User Interface Search. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 579, 22 pages. doi:10.1145/3706598.3714213

[19] Sungho Park, Joohyung Yun, Jongwuk Lee, and Wook-Shin Han. 2025. HELIOS: Harmonizing Early Fusion, Late Fusion, and LLM Reasoning for Multi-Granular Table-Text Retrieval. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Vienna, Austria, 2025-07), Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 32424–32444. doi:10.18653/v1/2025.acl-long.1559

[20] Maciej Pawłowski, Anna Wróblewska, and Sylwia Sysko-Romańczuk. 2023. Effective Techniques for Multimodal Data Fusion: A Comparative Analysis. 23, 5 (2023), 2381. doi:10.3390/s23052381

[21] Zhengyang Qu, Shahid Alam, Yan Chen, Xiaoyong Zhou, Wangjun Hong, and Ryan Riley. 2017. DyDroid: Measuring Dynamic Code Loading and Its Security Implications in Android Applications. 415–426. doi:10.1109/DSN.2017.14

[22] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>

[23] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. 3973–3983. doi:10.18653/v1/D19-1410

[24] Antu Saha, Yang Song, Junayed Mahmud, Ying Zhou, Kevin Moran, and Oscar Chaparro. 2024. Toward the Automated Localization of Buggy Mobile App UIs from Bug Descriptions. doi:10.1145/3650212.3680357

[25] Zewei Shi, Ruoxi Sun, Jieshan Chen, Jiamou Sun, Minhui Xue, Yansong Gao, Feng Liu, and Xingliang Yuan. 2025. 50 Shades of Deceptive Patterns: A Unified Taxonomy, Multimodal Detection, and Security Implications. In *Proceedings of the ACM on Web Conference 2025* (New York, NY, USA, 2025-04-22) (WWW '25). Association for Computing Machinery, 978–989. doi:10.1145/3696410.3714593

[26] Kushal Singla, Niloy Mukherjee, and Joy Bose. 2018. Multimodal Language Independent App Classification Using Images and Text. In *Natural Language Processing and Information Systems*, Max Silberstein, Faten Atigui, Elena Kornysheva, Elisabeth Métais, and Farid Meziane (Eds.). Vol. 10859. Springer International Publishing, 135–142. doi:10.1007/978-3-319-91947-8_14 Series Title: Lecture Notes in Computer Science.

[27] Kushal Singla, Niloy Mukherjee, Hari Koduvely, and Joy Bose. 2018. Evaluating Usage of Images for App Classification. doi:10.1109/INDICON45594.2018.8987165

[28] Cees G. M. Snoek, Marcel Worring, and Arnold W. M. Smeulders. 2005. Early versus Late Fusion in Semantic Video Analysis. In *Proceedings of the 13th Annual ACM International Conference on Multimedia (MULTIMEDIA '05)*. Association

- for Computing Machinery, New York, NY, USA, 399–402. doi:10.1145/1101149.1101236
- [29] Dewen Suo, Lei Xue, Weihao Huang, Runze Tan, and Guozi Sun. 2026. Assessing the capability of android dynamic analysis tools to combat anti-runtime analysis techniques. 234 (2026), 112747. doi:10.1016/j.jss.2025.112747
- [30] David H. Wolpert. 1992. Stacked Generalization. *Neural Networks* 5, 2 (1992), 241–259. doi:10.1016/S0893-6080(05)80023-1
- [31] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 <https://arxiv.org/abs/2309.07597> arXiv:2309.07597.
- [32] Yinglin Xie, Xinyi Hou, Yanjie Zhao, Kai Chen, and Haoyu Wang. 2025. LLM App Squatting and Cloning. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. Association for Computing Machinery, 64–74. <https://dl.acm.org/doi/10.1145/3696630.3728532>
- [33] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. 2024. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. In *Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part LXIV* (Berlin, Heidelberg, 2024-10-31). Springer-Verlag, 240–255. doi:10.1007/978-3-031-73039-9_14
- [34] Shengcheng Yu, Chunrong Fang, Ziyuan Tuo, Qianjun Zhang, Chunyang Chen, Zhenyu Chen, and Zhendong Su. 2023. Vision-Based Mobile App GUI Testing: A Survey. arXiv:2310.13518 doi:10.48550/arXiv.2310.13518 arXiv:2310.13518.
- [35] Yanjie Zhao, Xinyi Hou, Shenao Wang, and Haoyu Wang. 2025. LLM App Store Analysis: A Vision and Roadmap. 34, 5 (2025), 1–25. doi:10.1145/3708530

Unpublished working draft.
Not for distribution.